



STANDARD OPERATING PROCEDURE / WORK INSTRUCTION

คู่มือมาตรฐานการใช้งาน และเอกสารทางเทคนิค

User & Operator Manual + Technical Reference

ระบบควบคุมกระบวนการผสม **xMixing Control**

รหัสเอกสาร	MAN-PRD-MIX-MASTER-001
Revision	01 — Master Edition
วันที่บังคับใช้	25 มิถุนายน 2026
จัดทำโดย	xDev.CO,LTD
ลูกค้า (Client)	บริษัท มิตรผล จำกัด

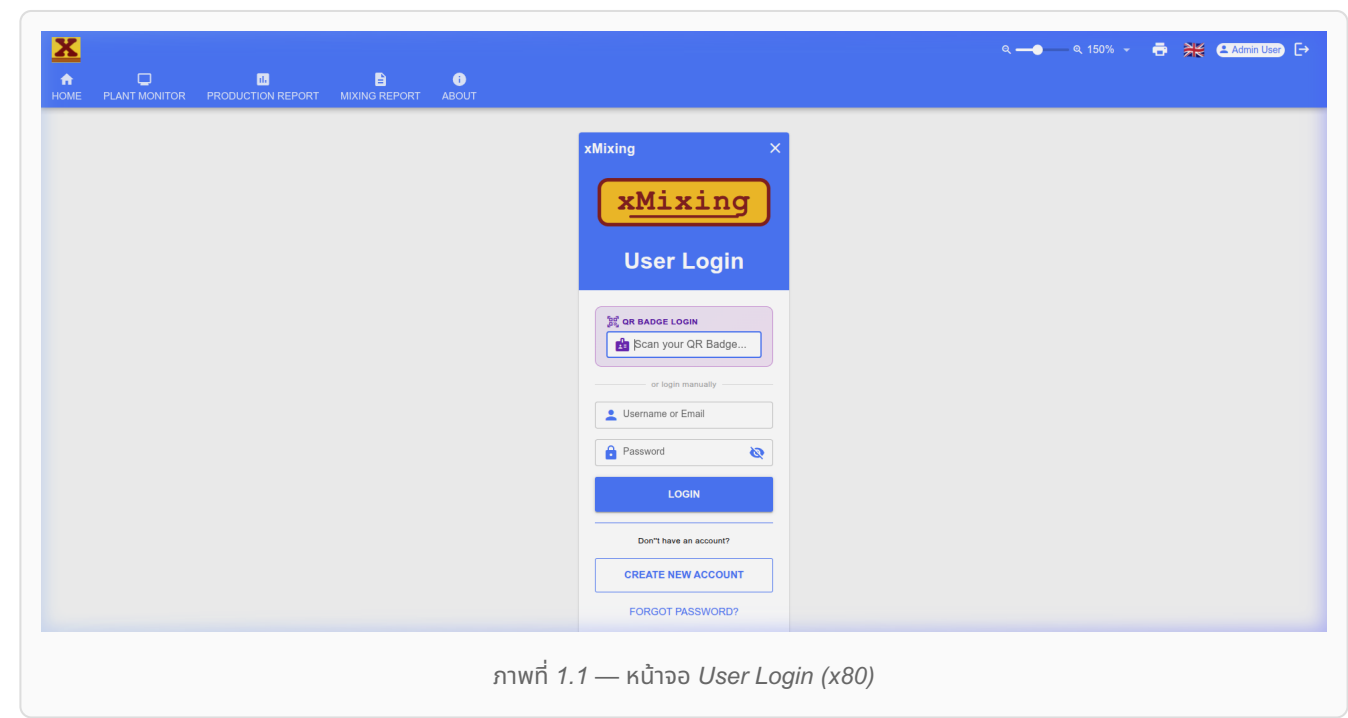
สารบัญ (Table of Contents)

บทที่ 1	คู่มือปฏิบัติงานพร้อมภาพประกอบ (Illustrated Work Instruction) Login Production Plan SKU View Check for Production Mixing Control Plant Monitor Report User Config Workflow สรุป
บทที่ 2	ขั้นตอนการสแกนและระบบป้องกันความผิดพลาด Barcode Scanning Logic Phase Guard Wrong Box FIFO
บทที่ 3	ภาพรวมการควบคุมกระบวนการผลิต Process Control Concept PLC Communication Flow
บทที่ 4	ระบบยืนยันขั้นตอน (Step Confirmation) Manual Dosing QC Brix-pH Confirmation Logic
บทที่ 5	คู่มือกู้คืนระบบกรณีฉุกเฉิน (PLC Recovery) Recovery Workflow Network Failure Restart Procedures
บทที่ 6	ข้อมูลจำเพาะฮาร์ดแวร์ — Siemens S7-1200 PLC DB100 Command Block DB200 Telemetry Block Watchdog Logic
บทที่ 7	แผนผังหน่วยความจำ PLC (Memory Map) DB1511 Recipe Array DB1512 Telemetry DB1513 Handshake
บทที่ 8	สถาปัตยกรรม PLC Recipe และข้อมูล SKU SKU Analysis (16 SKUs) Phase Types Payload Size
บทที่ 9	สถาปัตยกรรมระบบโดยรวม (System Architecture) Vue.js FastAPI Node-RED RabbitMQ PLC Integration
บทที่ 10	คู่มือการติดตั้งและ Deploy ระบบ (Deployment Guide) Network Setup Node-RED FastAPI PLC Configuration Checklist
บทที่ 11	ตารางสัญญาณเตือน (Alarm List) และการแก้ไขปัญหา Alarm List ทั้งหมด x60/x61 Troubleshooting Emergency Recovery Support Contacts

บทที่ 1: คู่มือปฏิบัติงานพร้อมภาพประกอบ (Illustrated Work Instruction)

1.1 การเข้าสู่ระบบ (User Login — x80)

วัตถุประสงค์: ยืนยันตัวตนพนักงานทุกครั้งก่อนเข้าใช้ระบบ ซึ่งจะถูกรับบันทึกลงทุก Batch Log



ภาพที่ 1.1 — หน้าจอ User Login (x80)

วิธีการใช้งาน: 1. เปิดแอปพลิเคชัน — ระบบจะแสดงหน้า Login อัตโนมัติ 2. **วิธีที่ 1 (สแกนบัตร):** ใช้ปิ่นสแกนยิงบัตรพนักงาน → กล้อง PIN จะปรากฏ → กรอก PIN 4 หลัก → กด **Login** 3. **วิธีที่ 2 (กรอกเอง):** พิมพ์ Username และ Password → กดปุ่ม **[เข้าสู่ระบบ]** 4. ชื่อพนักงานจะแสดงที่มุมขวาบนของทุกหน้า

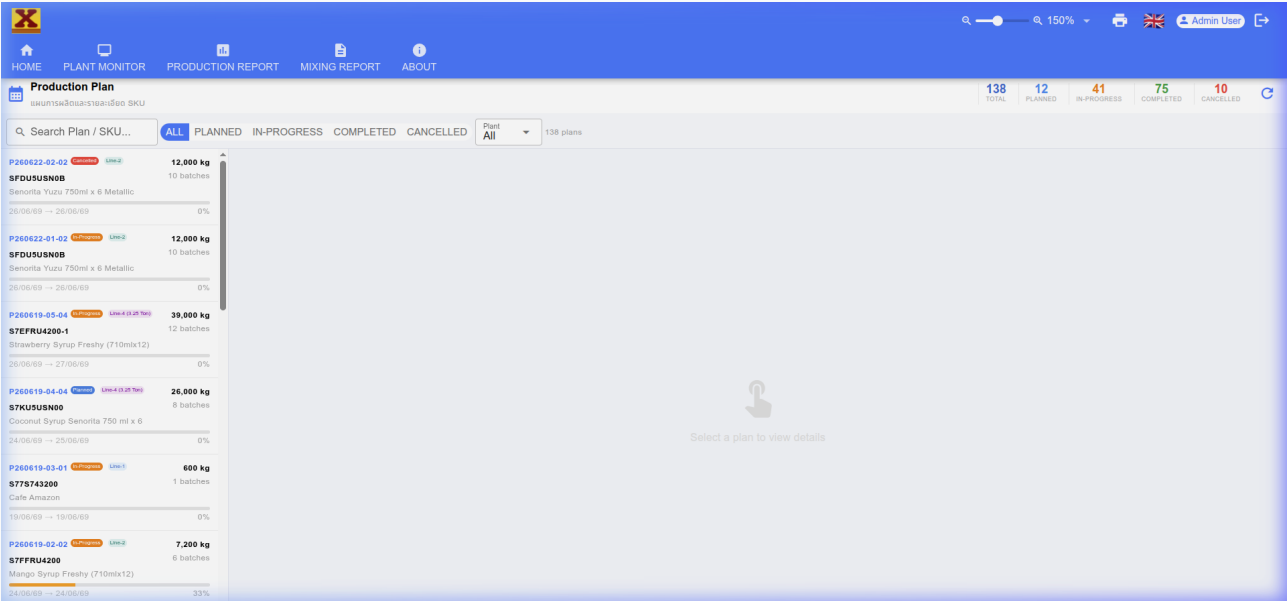
ปุ่มและการกระทำทั้งหมด:

ไอคอน / ปุ่ม	หน้าที่	เงื่อนไข
เข้าสู่ระบบ (Login)	ยืนยันตัวตนและเข้าระบบ	ต้องกรอก Username + Password ถูกต้อง
Login (Badge PIN)	ยืนยัน PIN หลังสแกนบัตร	กรอก PIN 4 หลักที่ตั้งไว้
สร้างบัญชีใหม่	ไปหน้า Register (x81)	เฉพาะ Admin เท่านั้น
Cancel	ยกเลิก PIN Dialog	ปิดกล้อง PIN กลับสู่หน้า Login
👁 (ดูรหัสผ่าน)	สลับแสดง/ซ่อนรหัสผ่าน	—

ไอคอน / ปุ่ม	หน้าที่	เงื่อนไข
✕ ปิด	ปิดหน้า Login	ต้อง Login ก่อนใช้งาน

1.2 แผนการผลิต (Production Plan — x55)

วัตถุประสงค์: ตรวจสอบรายการ Batch ที่รอผลิตและเลือก Batch ที่จะดำเนินการ



ภาพที่ 1.2 — หน้าจอแผนการผลิต (x55)

วิธีการใช้งาน: 1. กดเมนู **PRODUCTION PLAN** บนแถบด้านบน 2. ระบบแสดงตารางรายการ Batch ทั้งหมด เรียงตามวันที่ 3. กดคอลัมน์ **Status** — เลือก Batch ที่เป็นสีเขียว **Prepared** 4. กดปุ่ม **Select / เลือก** → ระบบโหลดข้อมูล Batch ไปยัง Mixing Control (x61)

คอลัมน์ในตาราง:

คอลัมน์	ความหมาย
Batch ID	รหัส Batch การผลิต เช่น P260622-01
SKU / Product Name	ชื่อสินค้า เช่น Senorita Yuzu 750ml
Batch Size (kg)	ปริมาณที่จะผลิต (กิโลกรัม)
Plant	เครื่องผสมที่จะใช้ (Plant 1/2/3)
Status	Created / Prepared / In-Progress / Done
วันที่ผลิต	วันที่วางแผนผลิต

ปุ่มและการกระทำ:

ปุ่ม	หน้าที่
Select (เลือก)	เลือก Batch นี้เพื่อเริ่มผลิต
Refresh	โหลดรายการแผนงานใหม่จาก Server

ความหมายสี Status Badge:

สี	สถานะ	ความหมาย
•	Created	สร้างแผนแล้ว รอเตรียมวัตถุดิบ
	In-Progress	กำลังเตรียมหรือกำลังผลิต
	Prepared	วัตถุดิบพร้อม — สามารถเดินเครื่องได้
	Done	ผลิตเสร็จสิ้น

1.3 สูตรการผลิต (SKU View — x56)

วัตถุประสงค์: ตรวจสอบสูตรและส่วนผสม (BOM) ของสินค้าแต่ละรายการก่อนผลิต

ภาพที่ 1.3 — หน้าจอสูตรการผลิต SKU View (x56)

วิธีการใช้งาน: 1. กดเมนู **SKU** บนแถบด้านบน 2. เลือกสินค้าจากรายการขายมือ หรือค้นหาชื่อ 3. ระบบแสดง Tab ต่างๆ: **Recipe Steps** (ขั้นตอน) และ **Ingredients** (วัตถุดิบ) 4. ตรวจสอบน้ำหนักและลำดับขั้นตอนก่อนเริ่มการผลิตจริง

ปุ่มและการกระทำ:

จัดทำโดย: xDev.CO, LTD

MAN-PRD-MIX-MASTER-001 | 25 มิถุนายน 2026

ลูกค้า: บริษัท มิตรผล จำกัด

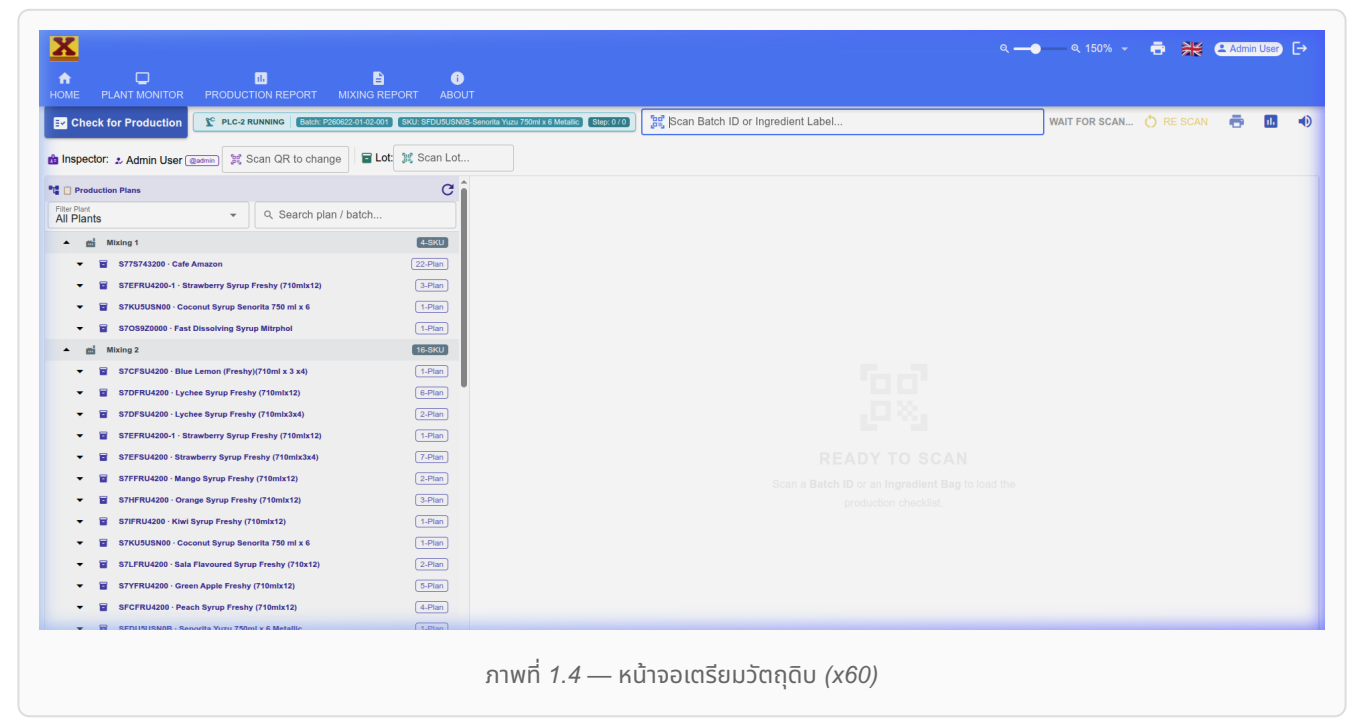
ปุ่ม	หน้าที่
Refresh	โหลดรายการสูตรใหม่
เลือก SKU	คลิกซื้อสินค้าเพื่อดูรายละเอียด

Tab ข้อมูลที่แสดง:

Tab	เนื้อหา
Recipe Steps	รายการขั้นตอน Phase/Step ทั้งหมด (Action Code, Phase, น้ำหนัก, อุณหภูมิ)
Ingredients	รายการวัตถุดิบ BOM ทั้งหมดพร้อมน้ำหนักรวม

1.4 การเตรียมวัตถุดิบ (Check for Production — x60)

วัตถุประสงค์: ชั่งตวงและยืนยันวัตถุดิบก่อนส่งเข้าสายการผลิต



วิธีการใช้งาน: 1. กดเมนู **CHECK FOR PRODUCTION** (ตราซึ่ง) 2. เลือก Batch ที่ต้องการเตรียมจากรายการซ้ายมือ (Tree สีน้ำเงิน=SPP / สีแดง=FH) 3. ชั่งน้ำหนักวัตถุดิบแต่ละรายการให้ได้ตาม **Require WT** 4. กดปุ่ม ✓ **Verify / Confirm** เมื่อน้ำหนักอยู่ใน Tolerance 5. เมื่อครบทุกรายการ → Batch Status เปลี่ยนเป็น **Prepared** (สีเขียว)

ปุ่มและการกระทำทั้งหมด:

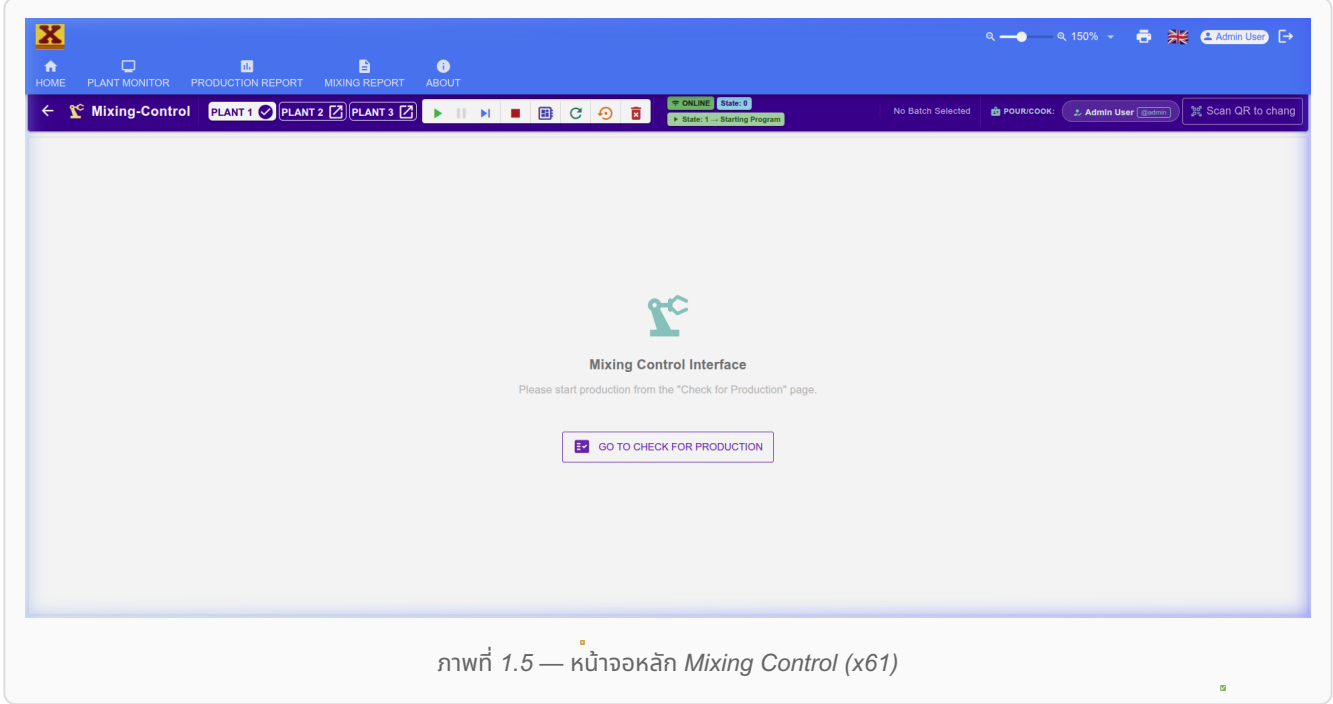
ไอคอน / ปุ่ม	หน้าที่	หมายเหตุ
✓ Verify Ingredient	ยืนยันวัตถุดิบรายการนั้น	กดได้เมื่อชั่งน้ำหนักแล้ว
Force Verify All (WH)	ยืนยันทั้งคลัง (SPP/FH) ที่เดียว	ต้องสิทธิ์ Supervisor
Print Batch Report	พิมพ์รายงานวัตถุดิบ Batch	ต้องเลือก Batch ก่อน
QC Report	เปิดรายงาน QC	—
Sound Settings	ตั้งค่าเสียง Alarm	—
Refresh	โหลดข้อมูลใหม่	—
× Clear Lot	ล้างรายการ Lot ที่เลือก	—
Hold Batch	พัก Batch ไว้ก่อน	กดที่รายการ Batch
GO TO MIXING	ข้ามไปหน้า Mixing Control (x61)	เมื่อ Prepared แล้ว

ความหมายสีแถบ Warehouse (WH):

สี	คลัง	ความหมาย
น้ำเงิน	SPP	วัตถุดิบจากคลัง SPP (High Shear / Specialty)
แดง	FH	วัตถุดิบจากคลัง FH (Flavor House)
เขียว	MIX	วัตถุดิบจากคลังผสม

1.5 การควบคุมการผสม (Mixing Control — x61)

วัตถุประสงค์: หน้าจอหลักสำหรับ Operator ควบคุมเครื่องผสม PLC และสแกนยืนยันวัตถุดิบทุกขั้นตอน



ภาพที่ 1.5 — หน้าจอหลัก Mixing Control (x61)

วิธีการใช้งาน (ขั้นตอนมาตรฐาน): 1. กดเมนู **MIXING CONTROL** (ไอคอนหน้าปิด) 2. กดเลือก **PLANT 1 / 2 / 3** ที่ต้องการควบคุม (แถบบนซ้าย) 3. ตรวจสอบไฟสถานะ PLC: **ONLINE** = พร้อม / **OFFLINE** = ขาดการติดต่อ 4. เลือก Batch จากรายการ (Status ต้อง Prepared) 5. กดปุ่ม **START** เพื่อเริ่มการผลิต 6. ระบบจะทำงานอัตโนมัติ — สังเกตแถบสีที่ Step ปัจจุบัน 7. เมื่อแถบเปลี่ยนเป็น **สีเหลือง (รอสแกน)** → ใช้ปืนสแกนยิง QR Code ที่ถุงวัตถุดิบ 8. ระบบยืนยัน → เดินเครื่องต่ออัตโนมัติ

ปุ่มควบคุมหลักทั้งหมด (Control Panel):

ไอคอน	ชื่อปุ่ม	หน้าที่	เงื่อนไข
■	START / RESUME	เริ่มหรือทำต่อหลัง Hold	PLC ต้อง Online
■	PAUSE / HOLD	หยุดเครื่องชั่วคราว PLC รักษาค่าเดิม	กำลัง Running
■	STOP / ABORT	ยกเลิก Batch ทันที (ฉุกเฉินเท่านั้น!)	เตือนยืนยันก่อน
■	NEXT STEP (Force)	บังคับข้าม Step ปัจจุบัน	ต้องสิทธิ์ Supervisor
	Refresh from PLC	โหลดสถานะจาก PLC ใหม่	—
	Print PDF	พิมพ์รายงานการผลิต	มีข้อมูล Batch
	PLC Data Block	ดู Memory ของ PLC ตรงๆ (Debug)	—
←	Back	กลับหน้าก่อน	—

ข้อมูลบนหน้าปิด (Gauges) ที่แสดง Real-time:

ค่า	ความหมาย	หน่วย
Temperature	อุณหภูมิถังผสมจริง	°C
Agitator RPM	ความเร็วใบกวน (Agitator) จริง	RPM
High Shear RPM	ความเร็ว High Shear จริง	RPM
Brix	ค่าความหวาน Brix จากเซ็นเซอร์	°Bx
pH	ค่า pH จากเซ็นเซอร์	—
Total Weight	น้ำหนักในถังผสมปัจจุบัน	kg
Timer	เวลาที่ใช้ใน Step ปัจจุบัน	นาที:วินาที

ความหมายสีแถบขั้นตอน (Step Row):

สี	ความหมาย	การกระทำของ Operator
เขียว	Active — PLC กำลังทำงาน Step นี้	เฝ้าดูค่าบนหน้าปัด
เหลือง	Waiting Scan — รอสแกนวัตถุดิบ	ใช้ปืนสแกน QR Code
เทา+	Completed — ทำเสร็จแล้ว	ไม่ต้องทำอะไร
แดง	Error / Tolerance Failed	แจ้งหัวหน้า ตรวจสอบค่า

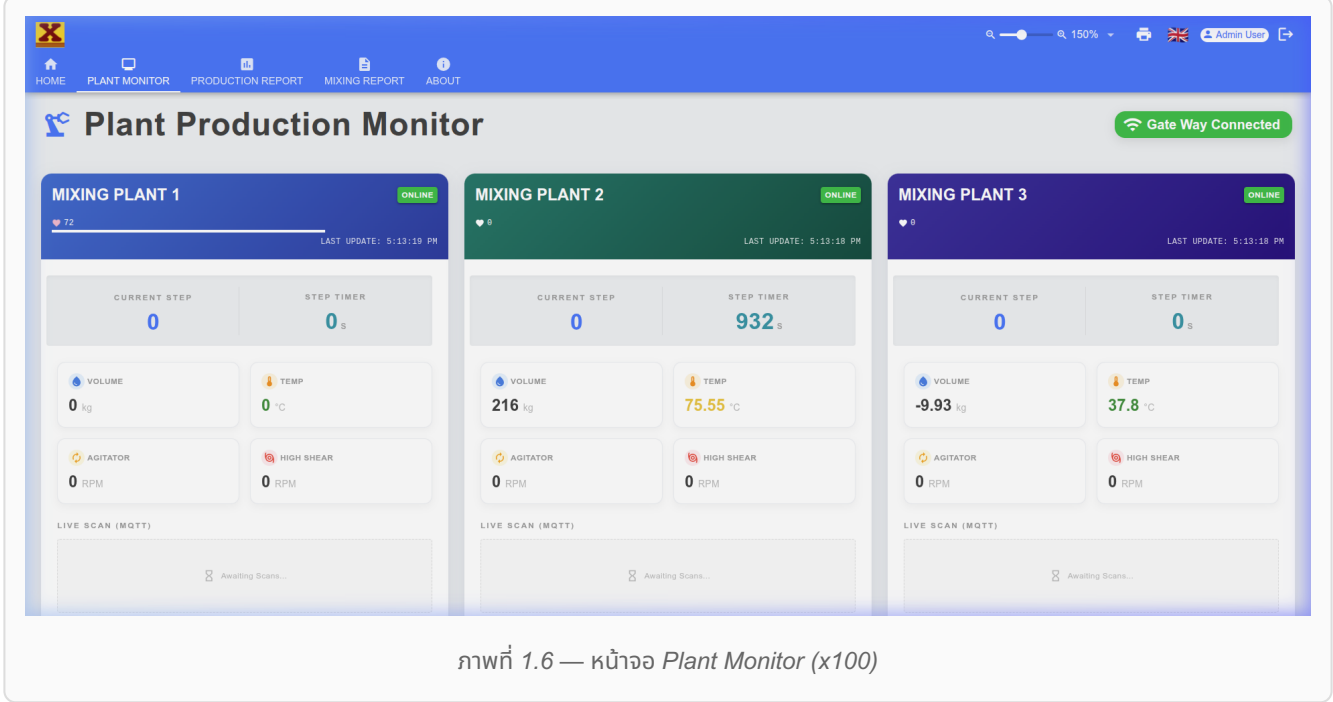
ระบบป้องกัน Phase Guard (Error Proofing):

กรณี	การแจ้งเตือน	สิ่งที่ต้องทำ
สแกนวัตถุดิบถูกต้อง	สีเขียว	ไม่ต้องทำอะไร ระบบดำเนินการต่อ
สแกนผิด Phase	Popup “สแกนผิด Phase!”	วางถุงไว้ก่อน หยิบถุงที่ถูกต้องมา
สแกนวัตถุดิบผิด Batch	“WRONG BOX!”	หยุดทันที! แจ้งหัวหน้า
สแกน Batch Label ซ้ำ	“สแกนหัวบิลซ้ำ!”	เปลี่ยนมาสแกนถุงวัตถุดิบแทน
Phase สแกนอิสระ (p010-p049)	หลังสแกนครบ	สแกนได้อิสระ แต่ต้องครบทุกตัวใน Phase

กรณีต้องกรอก QC (Brix / pH): 1. เครื่องหยุดรอที่ขั้นตอน QC Check 2. หน้าจอแสดงกล่องกรอกค่า 3. ดึง Sampling จากถัง → วัดค่า Brix และ pH 4. กรอกค่าลงในช่อง บนหน้าจอ 5. กด **Confirm QC** → เครื่องทำงานต่อ

1.6 ติดตามสถานะโรงงาน (Plant Monitor — x100)

วัตถุประสงค์: ดูภาพรวมเครื่องจักรทุก Plant พร้อมกัน เหมาะสำหรับหัวหน้ากะ



ภาพที่ 1.6 — หน้าจอ Plant Monitor (x100)

วิธีการใช้งาน: 1. กดเมนู **PLANT MONITOR** (ไอคอนโรงงาน) 2. ระบบแสดงการ์ดของแต่ละ Plant (1, 2, 3) พร้อมข้อมูล Real-time 3. คลิกที่การ์ด Plant เพื่อดูรายละเอียดหรือเข้าสู่หน้า Mixing Control

ข้อมูลที่แสดงในแต่ละการ์ด:

ข้อมูล		ความหมาย
Plant Name		ชื่อเครื่องผสม
PLC Status		Online / Offline
Current Step		ขั้นตอนที่กำลังทำอยู่
Batch ID		รหัส Batch ปัจจุบัน
Temperature		อุณหภูมิถังขณะนี้
Weight		น้ำหนักในถัง

ความหมายสีการ์ด:

สี	สถานะ
เขียว	Running — กำลังผลิต
เหลือง	Hold — หยุดพัก
แดง	Alarm / Error
เทา	Standby — ว่าง

1.7 รายงานการผลิต (Production Report — x70)

วัตถุประสงค์: ดูสรุปข้อมูลการผลิตและวัตถุดิบที่ใช้จริงย้อนหลัง

HOME

PLANT MONITOR

PRODUCTION REPORT

MIXING REPORT

ABOUT

Production Batch Records

รายงานการผลิตและวัตถุดิบทุกหน่วย

3383

15

674

957

123

ALL

DONE

IN-PROGRESS

PREPARED

CREATED

CANCELLED

HOLD

LAST 90D

1381 batches

P260622-01-02-001

Senorita Yuzu 750ml x 6 Metallic

1,200 kg

P260622-01-02

P260612-02-01-001

Cafe Amazon

1,200 kg

P260612-02-01

P260505-03-01-001

Coconut Syrup Senorita 750 ml x 6

1,200 kg

P260505-03-01

P260612-03-01-001

Strawberry Syrup Freshy (710mlx12)

1,200 kg

P260612-03-01

P260612-04-02-001

Blue Hawaii (Freshy)(710ml x 3 x4)

1,200 kg

P260612-04-02

P260427-02-02-001

Coconut Syrup Senorita 750 ml x 6

2,000 kg

P260427-02-02

P260601-03-02-001

Lychse Syrup Freshy (710mlx12)

1,200 kg

P260601-03-02

P260612-07-03-001

Classic caramel Syrup Senorita 750mlx6

500 kg

P260612-07-03

P260620-01-02-001

1,200 kg

Select a batch to view report

ภาพที่ 1.7 — หน้าจอรายงานการผลิต (x70)

วิธีการใช้งาน: 1. กดเมนู **PRODUCTION REPORT** 2. เลือก Batch จากรายการซ้ายมือ 3. ระบบแสดงรายละเอียดปริมาณวัตถุดิบจริง, เวลา, และผู้ปฏิบัติงาน

ปุ่มและการกระทำ:

ปุ่ม	หน้าที่
Refresh	โหลดรายการใหม่
Recent Only	แสดงเฉพาะ Batch ล่าสุด
Select Batch	เลือก Batch เพื่อดูรายงาน
Print	พิมพ์รายงาน
Export PDF	บันทึกเป็น PDF

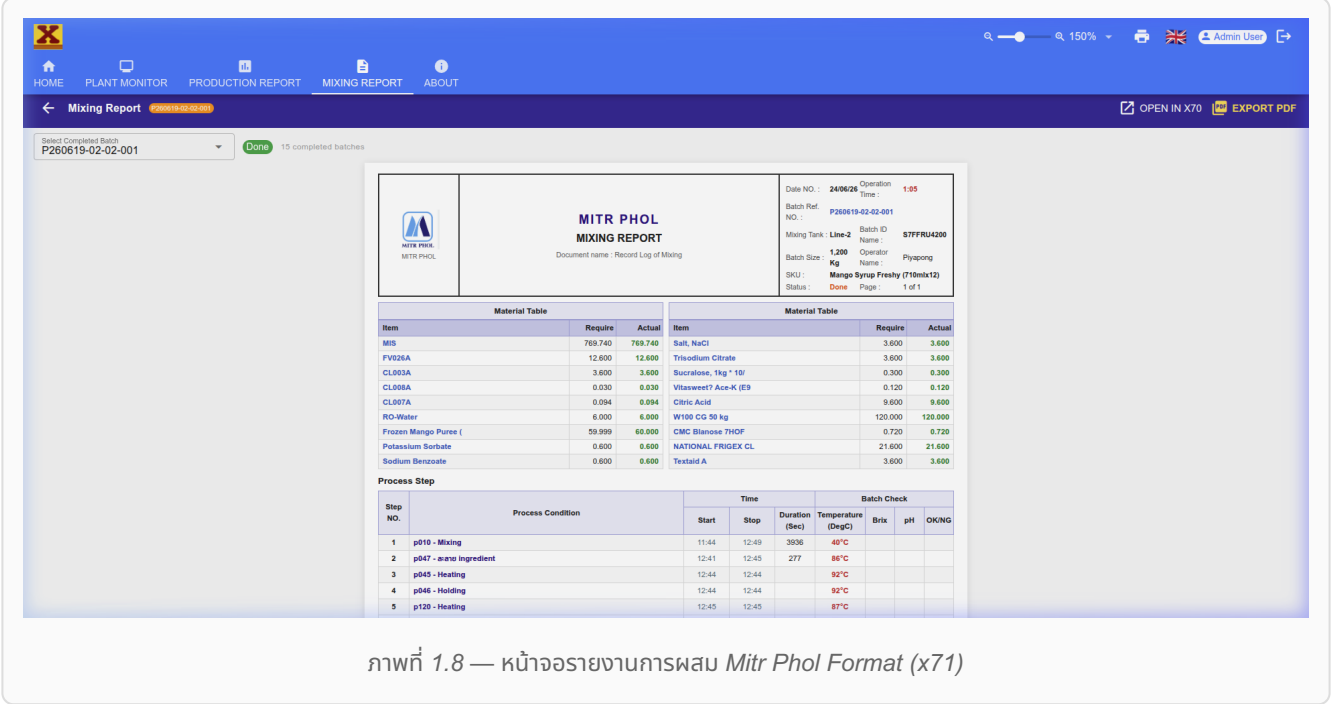
1.8 รายงานการผสม (Mixing Report — x71)

วัตถุประสงค์: รายงานแบบ Mitr Phol Format พร้อม Timeline การผสม, ค่า QC, และลายเซ็นผู้ปฏิบัติงาน

จัดทำโดย: xDev.CO, LTD

MAN-PRD-MIX-MASTER-001 | 25 มิถุนายน 2026

ลูกค้า: บริษัท มิตรผล จำกัด



ภาพที่ 1.8 — หน้าจอรายงานการผสม Mitr Phol Format (x71)

วิธีการใช้งาน: 1. กดเมนู **MIXING REPORT** 2. เลือก Batch ที่ต้องการออกรายงาน 3. ระบบแสดงรายงานพร้อม Logo Mitr Phol, ค่า Brix/pH ที่วัดได้จริง, เวลาเริ่ม-สิ้นสุดแต่ละ Step

1.9 การจัดการผู้ใช้งาน (User Config — x89)

วัตถุประสงค์: Admin จัดการบัญชีพนักงาน, สิทธิ์, และพิมพ์ QR บัตรพนักงาน

วิธีการใช้งาน: 1. เข้าเมนู **USER** → **User Config** (ต้องสิทธิ์ Admin) 2. ดูรายชื่อพนักงานทั้งหมด

ปุ่มและการกระทำ:

ปุ่ม	หน้าที่
+ สร้างผู้ใช้ใหม่	เพิ่มพนักงานและพิมพ์ QR Card
แก้ไข (ดินสอ)	แก้ไขข้อมูลหรือสิทธิ์ผู้ใช้
ลบ (ถังขยะ)	ลบบัญชีผู้ใช้
Print QR	พิมพ์บัตรพนักงาน QR Code
Toggle Permission	เปิด/ปิดสิทธิ์แต่ละด้าน

1.10 สรุปการทำงานทั้งระบบ (System Workflow Summary)

ลำดับขั้นตอนมาตรฐานตั้งแต่ต้นจนจบ Batch:

ลำดับ	ผู้รับผิดชอบ	หน้าจอ	การกระทำ
1	Admin / Planner	x55	สร้างแผนการผลิต (Batch Plan)
2	Prebatch Operator	x60	ตั้งตวงและ Confirm วัตถุดิบ → Status: Prepared
3	Mixing Operator	x61	เลือก Batch → กด START → คุ่มเครื่อง
4	Mixing Operator	x61	สแกน QR Code วัตถุดิบเมื่อระบบร้องขอ
5	Mixing Operator	x61	กรอกค่า Brix/pH เมื่อถึงขั้นตอน QC
6	Supervisor	x100	ติดตามสถานะทุก Plant
7	QC / Supervisor	x71	ออกรายงาน Mixing Report ฉบับ Mitr Phol

บทที่ 2: ขั้นตอนการสแกนและระบบป้องกันความผิดพลาด

Barcode Scanning & Verification Workflow

This diagram illustrates the step-by-step logic the application follows when an operator scans a QR code using the handheld scanner. It highlights the error-handling logic for various scanning mistakes, including the Wrong Box handling.

flowchart TD

A([Scanner reads QR Code]) --> B{Is a Batch already loaded?}

B -- No --> C[Load the scanned Batch]

B -- Yes --> D{Is scanned code a completely new Batch ID?}

D -- Yes --> E[Switch and Load the new Batch]

D -- No --> F[Verify as an Ingredient Bag]

F --> G[Check local data for matching ingredient]

G -- Match Found --> H([Mark Ingredient as Verified])

G -- No Local Match --> I[Send scan to Backend API]

I --> J{Backend Response}

J -- Success --> K([Mark Ingredient as Verified])

J -- Failure --> L{What did the operator actually scan?}

L -- "The same Batch ID again" --> M([Yellow Warning:
'You scanned the Batch Label!'])

L -- "Unrecognized Bag for this Batch" --> N([Red Toast Error:
'Ingredient not found'])

L -- "Bag from a completely different box" --> O([Big Red Modal:
WRONG BOX!])

O --> P{Operator Decision}

P -- "I want to keep working on my current batch" --> Q([Close Modal & Continue])

P -- "I want to scan a different batch instead" --> R([Start New Batch / Reset])

บทที่ 3: ภาพรวมการควบคุมกระบวนการ

Process Control Concept — xMixing Production

1. Process Flow Overview

จากการวิเคราะห์ SKU ทั้ง 16 ตัวจริงจากฐานข้อมูล พบรูปแบบการผลิตดังนี้:

```
graph TD
    subgraph "Phase A - Auto Batching"
        A1[A1010<br>LS/Water/Sugar<br>Batching 40°C]
        A2[A1020<br>CMC/Emulsifier<br>+ High Shear 60°C]
    end

    subgraph "Phase D - Dissolve & Mix"
        D1[D1010<br>ละลาย Ingredient<br>60-86°C]
        D3[D1030<br>เติมสารเคมี<br>Salt/Acid/Sweetener]
    end

    subgraph "Phase x - Transfer & Control"
        X1[x1010<br>Heating Up<br>86-92°C]
        X2[x1020<br>Holding Time<br>88°C x 300-900s]
        X3[x1030<br>Cooldown<br>75-78°C<br>Record Brix/pH]
        X4[x1040<br>Final Cooling<br>40-43°C<br>Record Brix/pH CTW]
    end

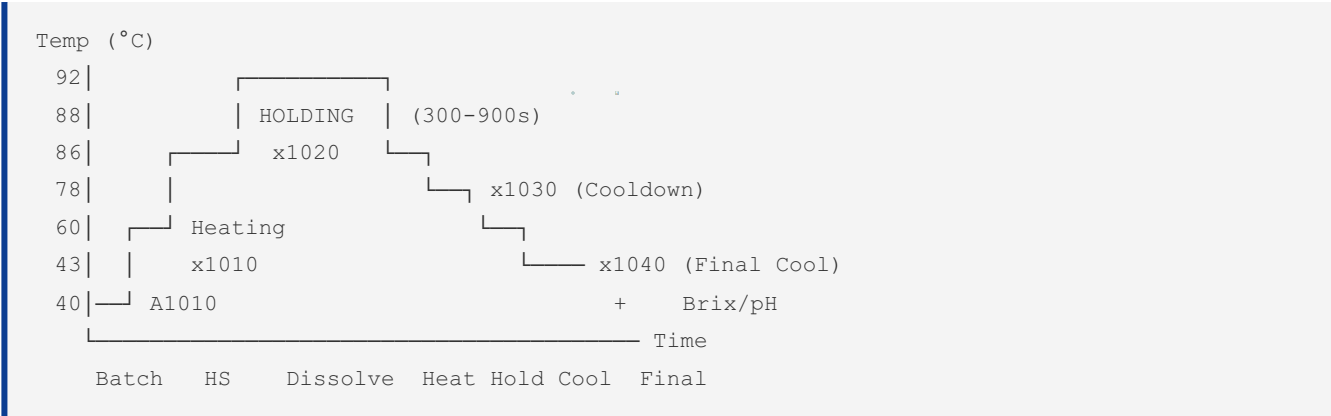
    A1 --> A2
    A2 --> D1
    D1 --> D3
    D3 --> X1
    X1 --> X2
    X2 --> X3
    X3 --> X4

    style A1 fill:#bbdefb
    style A2 fill:#bbdefb
    style D1 fill:#c8e6c9
    style D3 fill:#c8e6c9
    style X1 fill:#ffe0b2
    style X2 fill:#ffcdbb
    style X3 fill:#e1bee7
    style X4 fill:#b2ebf2
```

2. Temperature Zones (5 โซน)

Zone	Phase ID	อุณหภูมิ	Range	ระยะเวลา	วัตถุประสงค์
Mixing	A1010	40°C	-	ไม่กำหนด	ชั่งตวงวัตถุดิบหลัก (LS, Water, Sugar)
High Shear	A1020	60°C	-	120-180s	ผสม CMC, Emulsifier + High Shear 800-1700 RPM
Heating	x1010	86-92°C	85-95°C	ไม่กำหนด	ให้ความร้อนถึงจุดพาสเจอร์ไรส์
Holding	x1020	88°C	86-90°C	300-900s	คงอุณหภูมิฆ่าเชื้อ (Pasteurization)
Cooldown	x1030→x1040	78→43°C	40-80°C	ไม่กำหนด	ลดอุณหภูมิ + เติม Flavour + QC Check

Temperature Profile Chart:



3. Motor Control (2 ระบบ)

3.1 Agitator (ใบกวนหลัก)

สถานะ	RPM	ใช้ในช่วง
Low Speed	30 RPM	D1010, D1030 — ช่วงเติมวัตถุดิบละลาย
Normal Speed	60 RPM	A1020 — ช่วง High Shear ทำงาน
High Speed	80 RPM	A1010, x1010-x1040 — ช่วง Batch/Heat/Hold/Cool

3.2 High Shear (เครื่องบดละเอียด)

สถานะ	RPM	ใช้เมื่อ
OFF	0	ส่วนใหญ่ของกระบวนการ
Medium	800	เติมของเหนียว (Emulsifier, Textaid)
High	1700	เติม CMC, สารทำให้ข้น

[!IMPORTANT] High Shear ทำงานเฉพาะ Phase **A1020** เท่านั้น! และมี **timer control** (120-180 วินาที)

4. Recording & QC Checkpoints

4.1 ค่าที่ต้อง Record ตลอด (ทุก Step)

ค่า	Field	บันทึก	ประเภท
อุณหภูมิ	qc_temp	ทุก Step	Continuous
Steam Pressure	record_steam_pressure	ทุก Step	Continuous

4.2 ค่าที่ Record เฉพาะบาง Step (QC Checkpoint)

ค่า	Field	เมื่อไหร่	Phase ที่พบ
CTW (Cooling Tower Water)	record_ctw	ช่วง Final Cooling เท่านั้น	x1040
Brix	operation_brix_record	ช่วง Cooldown + Final	x1030, x1040
pH	operation_ph_record	ช่วง Cooldown + Final	x1030, x1040

4.3 QC Checkpoint Summary (จาก Blue Lemon Freshy 32 steps)

Step 1-23: ไม่มี Brix/pH record → เป็นขั้นตอนผลิตปกติ			
Step 24 (x1030 Cooldown 78°C):	Brix	pH	← QC Check #1
Step 25-31: ไม่มี			
Step 32 (x1040 Final 43°C):	Brix	pH	CTW ← QC Check #2 (Final)

5. Data Flow: Recipe → PLC → Record

```
sequenceDiagram
    participant DB as Central DB
    participant HMI as x31 Dashboard
    participant PLC as PLC Controller
    participant REC as Production Record DB

    Note over HMI: Operator clicks START
    HMI->>PLC: Download Recipe (ALL steps)

    loop Each Step
        PLC->>PLC: Execute step (dosing/heat/mix)
        PLC->>HMI: Telemetry (every 1s)<br>{temp, rpm, weight, timer}

        alt Step has record flag
            PLC->>HMI: Record Request<br>{actual_temp, actual_weight, actual_brix, actual_ph}
            HMI->>REC: Save Production Record
        end

        alt QC Checkpoint (x1030/x1040)
            PLC->>HMI: QC Data<br>{brix_actual, ph_actual, ctw}
            HMI->>HMI: Compare vs SP (Setpoint)
            HMI->>REC: Save QC Record
            Note over HMI: PASS or FAIL alert
        end
    end

    PLC->>HMI: Batch Complete
    HMI->>REC: Save Batch Summary
```

6. Production Record Schema

6.1 Per-Step Record (บันทึกทุก Step)

```
{
  "batch_id": "P260411-02-01-001",
  "step_no": 10,
  "phase": "p0010",
  "phase_id": "A1010",
  "re_code": "LS in Line",

  "setpoint": {
    "require": 371.71,
    "temperature": 40.0,
    "agitator_rpm": 80.0,
```

```
    "high_shear_rpm": 0
  },

  "actual": {
    "weight": 371.85,
    "temperature": 40.2,
    "agitator_rpm": 79.5,
    "high_shear_rpm": 0,
    "steam_pressure": 2.5,
    "elapsed_time": 45
  },

  "tolerance": {
    "weight_ok": true,
    "temp_ok": true
  },

  "timestamp_start": "2026-04-15T10:00:00",
  "timestamp_end": "2026-04-15T10:00:45",
  "recorded_by": "PLC"
}
```

6.2 QC Checkpoint Record (เฉพาะ x1030/x1040)

```
{
  "batch_id": "P260411-02-01-001",
  "checkpoint": "COOLDOWN",
  "phase_id": "x1030",

  "setpoint": {
    "temperature": 78.0,
    "temp_range": [75.0, 80.0],
    "brix_sp": "14.5",
    "ph_sp": "3.8"
  },

  "actual": {
    "temperature": 77.5,
    "brix": 14.3,
    "ph": 3.82,
    "ctw_temp": null,
    "steam_pressure": 2.1
  },

  "result": "PASS",
  "operator": "admin",
  "timestamp": "2026-04-15T10:15:30",
}
```

```
"notes": ""  
}
```

7. PLC Register Mapping Concept

Setpoint Registers (HMI → PLC) — Written per step

Register	Type	Description
DB100.0	INT	Step Number
DB100.2	INT	Action Code
DB100.4	REAL	Target Weight (kg)
DB100.8	REAL	Temp Setpoint (°C)
DB100.12	REAL	Temp Low Limit
DB100.16	REAL	Temp High Limit
DB100.20	REAL	Agitator RPM SP
DB100.24	REAL	High Shear RPM SP
DB100.28	INT	Step Time (s)
DB100.30	REAL	Low Tolerance
DB100.34	REAL	High Tolerance

Actual/Telemetry Registers (PLC → HMI) — Read every 1s

Register	Type	Description
DB200.0	INT	Current Step No
DB200.2	INT	Step Timer (s)
DB200.4	REAL	Mixing Tank Temp (°C)
DB200.8	REAL	Mixing Tank Weight (kg)
DB200.12	REAL	Agitator RPM Actual
DB200.16	REAL	High Shear RPM Actual
DB200.20	REAL	Hopper Weight (kg)
DB200.24	REAL	Steam Pressure

Register	Type	Description
DB200.28	REAL	CTW Temp
DB200.32	INT	Watchdog
DB200.34	INT	Status (0=IDLE, 1=RUN, 2=PAUSE, 3=DONE, 9=ERROR)

QC Registers (Operator Input or PLC Sensor)

Register	Type	Description
DB300.0	REAL	Actual Brix
DB300.4	REAL	Actual pH
DB300.8	REAL	Actual CTW Temp
DB300.12	BOOL	QC Record Trigger
DB300.13	BOOL	QC Pass/Fail

8. Implementation Plan

Phase 1: Recipe Download (Done)

- Send ALL recipe to PLC via MQTT
- PLC Simulator handles recipe
- Download button + ACK

Phase 2: Step-by-Step Telemetry (Current)

- PLC simulator uses recipe SP for realistic telemetry
- Dashboard highlights active step with actual vs SP
- Step auto-advance based on step_time

Phase 3: Production Records

- Create `production_records` table in DB
- API endpoint to save per-step records

- ☐
Auto-save actual values when step completes

Phase 4: QC Checkpoints

- ☐
Detect QC steps (brix/ph/ctw flags)
- ☐
Show QC input dialog at x1030/x1040
- ☐
Compare actual vs SP → PASS/FAIL
- ☐
Save QC records to DB

Phase 5: Batch Report

- ☐
Generate batch production report
- ☐
Include all step records + QC results
- ☐
Export as PDF for QA sign-off

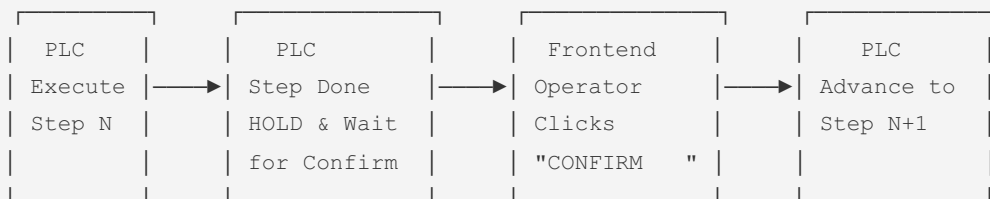
บทที่ 4: ระบบยืนยันขั้นตอน

Step Confirmation Concept — Operator Must Confirm Every Step

The Problem

In the current “full auto” design, the PLC auto-advances after timer/condition is met. But in real food manufacturing, the **operator must visually verify** each step is actually done before moving forward (e.g., “Is the sugar fully dissolved?”, “Did the temperature reach 83°C?”).

The New Workflow: “Execute → Hold → Confirm → Advance”



PLC State Machine (Updated)

State 0: IDLE	→ Waiting for recipe download + Start command
State 1: EXECUTING	→ Running current step (timer counting, outputs ON)
State 2: WAIT_CONFIRM	→ Step finished, holding outputs, waiting for operator
State 3: PAUSED	→ Operator paused the batch
State 4: DONE	→ All steps complete
State 9: ERROR	→ Fault condition

Key Change: After every step completes, PLC goes to **State 2 (WAIT_CONFIRM)** instead of directly advancing. The agitator/temperature HOLD at current setpoint while waiting, so the product is safe.

Data Block Updates (DB1780)

Add these fields to the header:

```
// — Confirmation Handshake —  
Step_Done      : Bool;    // PLC sets TRUE when step execution is finished  
Confirm_Step   : Bool;    // PC sets TRUE to acknowledge and advance
```

```
Confirm_Phase_ID    : String[10]; // PC echoes back which phase it's confirming
Confirm_Step_ID     : Int;        // PC echoes back which step it's confirming
```

PLC Logic Change (in FB1780):

```
// After step timer/condition is met:
IF step_finished THEN
    "DB_RecipeData".Step_Done := TRUE;
    "DB_RecipeData".PLC_State := 2; // WAIT_CONFIRM
    // DO NOT advance – hold current outputs
    // Agitator keeps running, temperature maintains
END_IF;

// When PC confirms:
IF "DB_RecipeData".Confirm_Step AND "DB_RecipeData".PLC_State = 2 THEN
    // Verify the confirmation matches current step (safety!)
    IF "DB_RecipeData".Confirm_Phase_ID = current_phase
        AND "DB_RecipeData".Confirm_Step_ID = current_sub_step THEN

        // Reset flags
        "DB_RecipeData".Step_Done := FALSE;
        "DB_RecipeData".Confirm_Step := FALSE;

        // ADVANCE to next step
        advance_to_next_step();
        "DB_RecipeData".PLC_State := 1; // Back to EXECUTING
    END_IF;
END_IF;
```

Frontend Changes (x62-MixingControlV2.vue)

What the Operator Sees:

Step 5/13 – Phase p0030, Sub-Step 10

Action: [20010] Dissolve Ingredient

Material: Potassium Sorbate

STEP EXECUTION COMPLETE

Temperature: 60.2°C / 60.0°C

Agitator: 1500 RPM

Duration: 5:00 / 5:00

CONFIRM STEP DONE & ADVANCE

Button Behavior:

- 1. Button is **DISABLED** while `PLC_State = 1` (Executing) — the step is still running
- 2. Button turns **GREEN** when `PLC_State = 2` (Wait_Confirm) — step is done, waiting for operator
- 3. Operator clicks → Frontend publishes MQTT:

```
{
  "Confirm_Step": true,
  "Confirm_Phase_ID": "p0030",
  "Confirm_Step_ID": 10,
  "timestamp": "2026-05-10T17:55:00"
}
```

- 4. PLC receives → validates → advances → `PLC_State` goes back to `1`
- 5. Frontend sees the state change and the next step starts highlighting

MQTT Message Flow

Step Execution (PLC → Frontend):

```
Topic: mixing/plant/1/status
{
  "PLC_State": 1,           ← Executing
  "Phase_ID": "p0030",
  "Step_ID": 10,
  "Step_Done": false,
  "Step_Timer": 245        ← Countdown
}
```

Step Ready for Confirm (PLC → Frontend):

```
Topic: mixing/plant/1/status
{
  "PLC_State": 2,           ← Wait Confirm
  "Phase_ID": "p0030",
  "Step_ID": 10,
  "Step_Done": true,        ← Step finished!
  "Step_Timer": 0
}
```

Operator Confirms (Frontend → PLC):

```
Topic: mixing/plant/1/step_confirm
{
  "Confirm_Step": true,
  "Confirm_Phase_ID": "p0030",
  "Confirm_Step_ID": 10
}
```

PLC Advances (PLC → Frontend):

```
Topic: mixing/plant/1/status
{
  "PLC_State": 1,           ← Back to Executing
  "Phase_ID": "p0030",
  "Step_ID": 20,           ← Next step!
  "Step_Done": false,
  "Step_Timer": 300
}
```

Safety Rules

Rule	Description
Phase/Step Echo	Frontend MUST echo back which Phase+Step it's confirming. PLC rejects if mismatch.
Double-click Guard	Frontend disables the confirm button for 2 seconds after click to prevent double-advance.
Heartbeat Still Active	If PC_Heartbeat is lost while in WAIT_CONFIRM, PLC holds position indefinitely (safe state).
Timeout Warning	If WAIT_CONFIRM exceeds 10 minutes, PLC can trigger a warning alarm (but does NOT auto-advance).
Manual Steps	For ActionCode 21010 (Manual Add), the confirm button serves double duty: operator scans the ingredient AND confirms.

Summary of Changes Required

Layer	What to Change
PLC DB1780	Add <code>Step_Done</code> , <code>Confirm_Step</code> , <code>Confirm_Phase_ID</code> , <code>Confirm_Step_ID</code> fields

Layer	What to Change
PLC FB1780	After step done → set State=2, hold outputs. On Confirm → validate → advance
Node-RED	Add S7-Out node to write <code>Confirm_Step</code> + IDs when MQTT confirm arrives
Frontend	Add “Confirm Step Done” button that enables only when <code>PLC_State = 2</code>
Mock Simulator	Update to wait for confirm MQTT message before advancing

บทที่ 5: คู่มือกู้คืนระบบฉุกเฉิน

PLC & PC Concept and Recovery Workflow

Architecture for seamless recovery from App or PC shutdown during mixing execution.

To handle scenarios where the App or Computer shuts down abruptly while maintaining the ability to seamlessly recover, the best approach is the “**PLC as Executor, PC as Supervisor**” concept.

Once the production starts, the PLC must not rely on the PC to feed it instructions step-by-step. Instead, the PLC holds the recipe and manages the state machine, while the PC just watches and logs.

1. The Concept (PLC as State Master)

- **Pre-load the Recipe:** The entire recipe (or up to 50 steps) is downloaded into the PLC's `DB_Recipe` before the batch officially starts.
- **PLC Manages the Sequence:** The PLC uses an internal variable (e.g., `Current_Step_Idx`) to track where it is. It executes Step 1, finishes it, and automatically moves to Step 2 without asking the PC.
- **PC is just a Viewer/Logger:** The App continuously polls the PLC to read `Current_Step_Idx` and logs the actual values (actual temp, time taken, etc.) to the database.
- **Resume Capability:** Because the PLC knows the `Batch_ID` and the `Current_Step_Idx`, if the PC reboots, the PC simply asks the PLC: “What Batch are you running, and what step are you on?” and immediately syncs its UI back to reality.

2. Required Updates to your Data Block Header

To make recovery work, you need to add a few synchronization variables to your `DB_Recipe` header:

```
// Header Information
Batch_ID      : STRING[20];    // Critical: Links PLC memory to your Database Batch
SKU_ID        : STRING[20];
Total_Steps   : INT;
Current_Step_Idx : INT;        // PLC updates this as it moves forward

// Handshake & Status
Command_Start  : BOOL;        // PC tells PLC to Start
Command_Pause  : BOOL;        // PC tells PLC to Pause
PC_Heartbeat   : BOOL;        // Toggles every 1s. If PLC doesn't see toggle, PC is
dead.
PLC_State      : INT;          // 0=Idle, 1=Running, 2=Waiting for Manual, 3=Paused,
4=Done
```

3. Normal Workflow (No Crashes)

1. **Download:** Operator selects a Batch. The App writes the `Batch_ID` and the array of up to 50 steps to the PLC.
2. **Start:** App sends `Command_Start = TRUE`.
3. **Execution:** PLC reads `Steps[Current_Step_Idx]`, sets Agitator, High Shear, and Temp. It waits for the `Step_Time` to finish.
4. **Transition:** When the time is up, PLC increments `Current_Step_Idx += 1` and executes the next step automatically.
5. **Logging:** The App notices `Current_Step_Idx` changed, so it records the completion of the previous step into your `mixing_batch_step_log` table.

4. Recovery Workflow (App / Computer Crashes & Restarts)

Scenario: The PC dies during Step 15.

1. PLC Reaction (PC_Heartbeat fails):

- The PLC detects the `PC_Heartbeat` stopped toggling.
- **If the step is automatic** (e.g., mixing for 5 minutes), the PLC *continues* running the step to avoid ruining the product.
- **If the step requires manual scanning** (like Action 21010 “Manual Add”), the PLC safely goes into a `Waiting` state (keeping agitator on, but timer paused) because the operator needs the PC scanner to continue.

2. PC Reboots & App Starts:

- The App connects to the PLC and reads the `Batch_ID` and `Current_Step_Idx`.
- **App Logic:** “Ah, the PLC says it is running Batch `B20260510-01` at Step `15`.”

3. Syncing the Database:

- The App checks its database for `B20260510-01`.
- It sees the batch status is still `In Progress`.
- It sees logs up to Step 14. It knows Step 15 is currently active.

4. Resume UI:

- The App instantly routes the operator to the Mixing Control screen for that Batch, highlights Step 15 as “Running”, and resumes polling data. **Recovery Complete without any operator input.**

5. PLC Communication Protocol: The Best Choice

Since your architecture already utilizes **Node-RED** and **RabbitMQ**, the absolute most robust way to communicate with the PLC is to decouple the PC from the PLC physically using Node-RED as middleware.

Option A: S7 Protocol via Node-RED (Recommended for Siemens S7-1200/1500) * Use the `node-red-contrib-s7` node. * **Why:** It natively reads/writes to the Data Block (DB) based on absolute offsets (e.g., `DB1, X0.0`) without requiring any special code or licenses on the PLC side. It is extremely fast and lightweight.

Option B: OPC UA (Recommended if PLC supports it natively) * Use `node-red-contrib-opcua`. * **Why:** Tag-based and highly standardized. If the PLC tags move, OPC UA doesn't break.

The Ideal Communication Flow: 1. **Polling:** Node-RED connects to the PLC and polls the `DB_Recipe` header every 500ms. 2. **Publishing:** If `Current_Step_Idx` or `PLC_State` changes, Node-RED publishes a JSON message to a **RabbitMQ** topic (e.g., `plc.mixing.state`). 3. **Consuming:** FastAPI listens to RabbitMQ, logs the step completion securely into MySQL (`mixing_batch_step_log`), and broadcasts the live state to Nuxt.

6. Required Changes to Existing Applications

Backend (FastAPI) Updates

1. **Remove State-Machine Logic:** Remove any Python logic that says “Wait 5 minutes then trigger the next step”. FastAPI must become stateless; it only reacts to events emitted by the PLC via RabbitMQ.
2. **Add Download Endpoint:** Create a `POST /api/batch/{id}/download-to-plc` endpoint. This query formats the 50-step array and sends it to Node-RED to write to the PLC `DB_Recipe`.
3. **RabbitMQ Consumer Worker:** Create a background worker in FastAPI to consume PLC state changes from RabbitMQ and update the MySQL database instantly.

Frontend (Nuxt) Updates

1. **Make UI Stateless:** The UI must stop driving the process. Remove the manual “Next Step” buttons for any automated mixing/heating steps.
2. **WebSocket / SSE Integration:** The frontend should connect to FastAPI via WebSocket. When the PLC moves to Step 15, the frontend instantly receives the event and visually highlights Step 15 in the UI.
3. **Recovery on Mount (onMounted):** When the operator navigates to the Mixing Control page, the code should immediately fetch the current `PLC_State` and `Current_Step_Idx`. If the batch is already running, the UI simply jumps to that step and resumes monitoring.
4. **Heartbeat Signal:** The Nuxt frontend (or FastAPI) must implement a loop that toggles the `PC_Heartbeat` bit in the PLC every 1-2 seconds so the PLC knows the PC is alive.

บทที่ 6: ข้อมูลจำเพาะฮาร์ดแวร์ PLC S7-1200

S7-1200 PLC Data Block & Interaction Design

Method: Step-by-Step Handshake (via Node-RED & MQTT)

This document defines the data structures (Data Blocks) used for real-time communication between the HMI/App and the Siemens S7-1200 PLC.

DB100: Step Command (App → PLC)

Role: Receives recipe parameters and execution commands for the current step. **Access:** App writes (via MQTT/Bridge), PLC reads.

Name	Data Type	Description	Example
Watch_Doc	Int	Watchdog timer/counter from App	1234
Plan_ID	String[20]	Production Plan Identifier	'PLAN-2024-001'
Batch_ID	String[20]	Current Production Batch	'P260411-02-01'
SKU_Name	String[30]	Product name or SKU identifier	'Strawberry_Jam_01'
Phase_ID	String[10]	Phase identifier code	'A1010'
Step_ID	Int	Unique Step ID	501
Step_Time_SP	Int	Target Dwell/Mixing time (seconds)	600
Step_Status	Int	0=Pending, 1=Active, 2=Complete	1
Material_ID	String[20]	Specific Raw Material ID	'MAT-099'
Re_Code_ID	String[20]	Ingredient or Recipe Code	'RM-SUG-01'
Req_Qty	Real	Required Quantity for step	250.5
TT_SP	Array[0..16] of Real	Temperature Profile Setpoints	[60.0, 65.0, ...]
Agitator_Speed	Real	Target Agitator Speed (RPM)	30.0
High_Shear_SP	Real	Target High Shear Speed (RPM)	1500.0
PH_Target	Real	Target pH level	4.5

Name	Data Type	Description	Example
Brix_Target	Real	Target Brix value	12.5
HMI_Command	Int	0=IDLE, 1=START, 2=PAUSE, 3=ABORT	1
Cmd_NewStep	Bool	Trigger Flag to start the step	TRUE

DB200: Telemetry (PLC → App)

Role: Live monitoring and feedback from the physical hardware. **Access:** PLC writes, App reads (polled or streamed every 1s).

Name	Data Type	Description	Example
Watchdog	Int	Heartbeat signal from PLC	32767
PLC_State	Int	0=Ready, 1=Run, 2=Hold, 9=Error	1
Current_Step	Int	Which step the PLC is actively executing	5
Step_Timer	Int	Elapsed time in current step (seconds)	124
Step_Status_Act	Int	Current Step Phase (e.g., 0=Init, 1=Dosing, 2=Mixing)	2
MixTank_Temp	Real	Live Tank Temperature (°C)	59.5
MixTank_Weight	Real	Live Tank Weight (kg)	249.8
Agitator_Act	Real	Live Agitator Speed (RPM)	30.1
HighShear_Act	Real	Live High Shear Speed (RPM)	1498.2
PH_Actual	Real	Live pH Sensor Reading	4.52
Brix_Actual	Real	Live Brix Sensor Reading	12.4
Hopper_Weight	Real	Live Sub-Hopper Weight (kg)	0.0

Interaction Flow (Handshake)

The communication follows a strict handshake protocol to ensure safety in manual and automatic modes.


```
sequenceDiagram
    participant App as Web Application
    participant NR as Node-RED / Bridge
    participant DB100 as PLC DB100 (CMD)
    participant DB200 as PLC DB200 (STATUS)

    Note over App,DB200: Step Execution Flow

    App->>NR: topic: plc/command (Step Data)
    NR->>DB100: Write All Params + Cmd_NewStep = 1

    Note over DB100: PLC Detects NewStep Trigger
    DB100->>DB100: Logic Starts Step
    DB100->>DB100: PLC Reset Cmd_NewStep = 0 (ACK)

    loop Every 1s
        DB200-->>NR: Current_Step, Temp, Timer
        NR-->>App: UI Update (Live Dashboard)
    end

    Note over DB200: PLC Finished Step
    DB200->>App: Step_Status_Act = Complete
```

Implementation Guidance

- Watchdog Logic:** Both the App (DB100) and PLC (DB200) increment their watchdog values. If a side detects that the other side's value hasn't changed for >5 seconds, it should trigger a **Communication Error** and safe-stop any active motor/pump.
- String Handling:** S7 strings use the first two bytes for length metadata. The App serialization must account for this (2 bytes + actual string data).
- Trigger Reset:** The PLC should reset `Cmd_NewStep` to `FALSE` as soon as it acknowledges the new parameters, serving as a handshake confirmation back to the App.

บทที่ 7: แผนผังหน่วยความจำ PLC

DB1511 Memory Map (S7-1200 Non-Optimized)

This document maps the exact byte offsets for **DB1511 (DB_FULL_RECIPE)**. This is extremely important for writing your `python-snap7 serialize()` function, as you must pack the binary payload to match these exact byte locations.

Important: This assumes the DB is set to `S7_Optimized_Access := 'FALSE'` (Standard Access) so offsets are deterministic.

1. `type_FullRecipe` (Header & Array)

This is the top-level structure of DB1511. The header takes up the first 52 bytes, followed by the array of 128 steps.

Offset	Name	Data Type	Size (Bytes)	Description
+0.0	Batch_ID	String[20]	22	Batch Code (e.g. "P260411-02"). S7 strings have 2 header bytes.
+22.0	SKU_ID	String[20]	22	Recipe/SKU Code
+44.0	HMI_Command	Int	2	0=IDLE, 1=START, 2=PAUSE, 3=ABORT, 9=RESET
+46.0	Total_Steps	Int	2	How many steps are actually populated (1-128)
+48.0	Active_Step	Int	2	PLC Pointer — current Seq number (1-128)
+50.0	Cmd_LoadRecipe	Bool	1 (bit 0)	Trigger bit to tell PLC a new array is ready
padding	(Padding)	(Byte)	1	

Offset	Name	Data Type	Size (Bytes)	Description
				S7 aligns Arrays to even byte boundaries
+52.0	Steps[1]	type_RecipeStep	78	First step (See Table 2)
+130.0	Steps[2]	type_RecipeStep	78	Second step
+208.0	Steps[3]	type_RecipeStep	78	Third step
...	...	...	78	... repeats ...
+9958.0	Steps[128]	type_RecipeStep	78	Last step

(Total Size of DB1511: **10,036 bytes** ≈ 10 KB)

2. type_RecipeStep (Individual Step Structure)

Each element in the Steps[1..128] array is exactly **78 bytes** long.

Relative Offset	Name	Data Type	Size (Bytes)	Description
+0.0	Seq	Int	2	Flat sequence number (1, 2, 3, ... 128)
+2.0	Phase_No	Int	2	Phase number (10, 20, 30...)
+4.0	Sub_Step	Int	2	Step within phase (10, 20, 30...)
+6.0	Action_Code	String[10]	12	Action code (e.g. "x10010")
+18.0	Phase_ID	String[10]	12	Phase ID (e.g. "p0010")
+30.0	Re_Code	String[20]	22	Ingredient / Material code
+52.0	Target_Weight	Real	4	Target dosing weight (kg)
+56.0	Temp_SP	Real	4	Temperature setpoint (°C)

Relative Offset	Name	Data Type	Size (Bytes)	Description
+60.0	Temp_Low	Real	4	Min temperature limit (°C)
+64.0	Temp_High	Real	4	Max temperature limit (°C)
+68.0	Agitator_SP	Real	4	Agitator speed (RPM)
+72.0	HighShear_SP	Real	4	High shear speed (RPM)
+76.0	Step_Time	Int	2	Hold time (Seconds)

(Total Size per Step: 78 bytes)

3. Example: Cafe Amazon (S77S743200) — Flattened into Array

This is how the database recipe for SKU “Cafe Amazon” would be flattened into the `Steps[1..128]` array:

Seq	Phase_No	Sub_Step	Action_Code	Phase_ID	Ingredient	Target (kg)
1	10	10	x10010	p0010	LS in Line	371.71
2	10	20	x10020	p0010	RO-Water	372.87
3	10	30	x10030	p0010	White Sugar W150	250.0
4	20	10	x20010	p0020	(empty)	0.0
5	30	10	x30010	p0030	Potassium Sorbate	0.8
6	30	20	x30020	p0030	Sodium Benzoate	0.2
7	30	30	x30030	p0030	Malic Acid	0.115
8	30	40	x30040	p0030	Caramel Colour III	0.2
9	30	50	x30050	p0030	RO-Water	4.0
10	40	10	x40010	p0040	(empty)	0.0
11	45	10	x45010	p0045		0.11

Seq	Phase_No	Sub_Step	Action_Code	Phase_ID	Ingredient	Target (kg)
					Sugar Flavour SG-01	
12	50	10	x50010	p0050	(empty)	0.0
13	60	10	x60010	p0060	(empty)	0.0
14-128	0	0			(unused — Seq=0)	0.0

The PLC reads `Total_Steps = 13` , so it knows to stop after Seq 13 and ignore Steps[14] to Steps[128].

Python Serialization Tip

When you write the `.serialize()` function in `plc_interface.py`:

```
import struct

def pack_s7_string(s: str, max_len: int) -> bytes:
    s_bytes = s.encode('ascii')[:max_len]
    return struct.pack('BB', max_len, len(s_bytes)) + s_bytes.ljust(max_len, b'\x00')

# Pack Header (52 bytes)
header = b""
header += pack_s7_string(batch_id, 20)           # +0
header += pack_s7_string(sku_id, 20)             # +22
header += struct.pack('>h', hmi_command)         # +44
header += struct.pack('>h', total_steps)          # +46
header += struct.pack('>h', active_step)          # +48
header += struct.pack('>?', cmd_load_recipe)      # +50
header += b'\x00'                                # +51 padding

# Pack Each Step (78 bytes each)
for step in steps:
    header += struct.pack('>hhh', step.seq, step.phase_no, step.sub_step) # +0,2,4
    header += pack_s7_string(step.action_code, 10) # +6
    header += pack_s7_string(step.phase_id, 10)    # +18
    header += pack_s7_string(step.re_code, 20)     # +30
    header += struct.pack('>ffffff',
        step.target_weight, step.temp_sp, step.temp_low,
        step.temp_high, step.agitator_sp, step.highshear_sp) # +52..+76
    header += struct.pack('>h', step.step_time)    # +76
```

```
# Total payload = 52 + (128 * 78) = 10,036 bytes  
plc.db_write(db_number=1511, start=0, data=header)
```

บทที่ 8: สถาปัตยกรรม Recipe และข้อมูล SKU

SKU Recipe Analysis — All 16 Active SKUs

1. Full SKU Breakdown

SKU ID	Product Name	Steps	Phases	Max Steps/Phase	Avg Steps/Phase
S77S743200	Cafe Amazon	13	7	5	1.9
SFMU5USN00	Fresh Mint Senorita 750ml	15	8	4	1.9
S7GCA4SNA0	Japanese Melon Senorita 1900ml	17	8	8	2.1
S7GU5USN00	Japanese Melon Senorita 750ml	17	8	8	2.1
S7KCY4SNA0	Coconut Senorita 1900ml	17	9	4	1.9
S7KU5USN00	Coconut Senorita 750ml	17	9	4	1.9
S77CY4SN00	Classic Caramel Senorita	18	9	6	2.0
SFLDBUSS00	Rose Senorita Signature 720ml	24	10	7	2.4
S7EFRU4200-1	Strawberry Freshy 710ml x12	30	15	5	2.0
S7EFSU4200	Strawberry Freshy 710ml x3	30	15	5	2.0
S7HFRU4200	Orange Freshy 710ml x12	31	12	15	2.6
S7CFSU4200	Blue Lemon Freshy 710ml x3	32	17	5	1.9
S7DFRU4200	Lychee Freshy 710ml x12	34	16	5	2.1
S7DFSU4200		34	16	5	2.1

SKU ID	Product Name	Steps	Phases	Max Steps/Phase	Avg Steps/Phase
	Lychee Freshy 710ml x3				
S7YFRU4200	Green Apple Freshy 710ml x12	34	18	5	1.9
SFGFSU4200	Blue Hawaii Freshy 710ml x3	35	17	5	2.1

2. Statistical Summary

16 SKUs Analyzed			
Metric	Min	Max	Avg
Steps per SKU	13	35	24.9
Phases per SKU	7	18	12.1
Max steps in 1 phase	-	15	-
Avg steps per phase	1.9	2.6	2.1

3. Phase Type Classification

พบ 3 ประเภท Phase ในทุก SKU:

Prefix	Type	หน้าที่	ตัวอย่าง
A	Auto Batching	ซึ่งตรวจวัดวัตถุดิบอัตโนมัติ (น้ำ, น้ำตาล, LS)	A1010, A1020
D	Dissolve / Mix	ละลาย, ผสม, ต้ม, เติมน้ำวัตถุดิบมือ	D1010, D1030
x	Transfer / Control	โอนย้ายถัง, รอ, QC, จบกระบวนการ	x1010, x1020, x1030, x1040

Process Flow Pattern (พบในทุก SKU):

```
graph LR
  A[A - Auto Batching<br>ซึ่งตรวจวัดวัตถุดิบอัตโนมัติ] --> X1[x - Transfer<br>โอนเข้าถังผสม]
  X1 --> D[D - Dissolve<br>ละลาย/ผสม]
  D --> X2[x - Transfer<br>โอนออก]
  X2 --> X3[x1020 - QC Check]
```


X3 --> X4[x1030 - Storage]
X4 --> X5[x1040 - Complete]

4. Payload Size Analysis

Method	Worst Case	Typical	Comment
Send ALL	35 steps × 200B = 6.8 KB	25 × 200B = 4.9 KB	เล็กมาก!
Send per Phase	15 steps × 200B = 2.9 KB	2 × 200B = 0.4 KB	เล็กเกินไป
Send per Step	1 × 200B = 0.2 KB	-	ไม่คุ้ม overhead

[!!IMPORTANT] ข้อมูลสำคัญ: Payload ของสูตรที่ใหญ่ที่สุด (35 steps) มีขนาดเพียง **6.8 KB** เท่านั้น — เล็กมากๆ สำหรับ PLC สมัยใหม่ที่มี memory เป็น MB ขึ้นไป

5. คำแนะนำ: Send ALL (Option A)

เหตุผล:

#	เหตุผล	รายละเอียด
1	Payload เล็กมาก	แม้ SKU ที่ซับซ้อนสุด (35 steps) ก็แค่ 6.8 KB — PLC รับได้สบายๆ
2	Avg 2.1 steps/phase	ถ้าส่งทีละ Phase จะต้อง handshake 12-18 ครั้ง เปลืองเกินไป
3	PLC ทำงานอิสระ	ไม่ต้องรอ Network ระหว่างขั้นตอน — ปลอดภัยกว่า สำหรับโรงงาน
4	ง่ายต่อการ Debug	ส่งครั้งเดียว ตรวจสอบว่าครบถ้วน แล้ว PLC รันต่อเอง
5	Network Failure Safe	ถ้าเน็ต MQTT หลุดระหว่างผลิต PLC ยังวิ่งต่อได้

Architecture ที่แนะนำ:

```
sequenceDiagram
    participant OP as Operator
    participant HMI as x31 Dashboard
    participant MQTT as MQTT Broker
    participant PLC as PLC

    OP->>HMI: Click "Start Production"
    HMI->>HMI: Fetch all SKU steps from DB
```

```
HMI->>MQTT: mixing/plant/1/recipe<br>{batch_id, sku_id, batch_size,<br>steps}}
MQTT->>PLC: Full recipe (max 6.8 KB)
PLC->>MQTT: mixing/plant/1/ack<br>{status: "RECIPE_LOADED", steps: 35}

Note over HMI:      Recipe confirmed, PLC ready
OP->>HMI: Click "START"
HMI->>MQTT: mixing/plant/1/cmd<br>{command: "START"}

loop PLC executes autonomously
  PLC->>MQTT: mixing/plant/1/telemetry<br>{step_no, temp, rpm, weight, watchdog}
  HMI->>HMI: Update gauges in real-time
end

PLC->>MQTT: mixing/plant/1/status<br>{status: "BATCH_COMPLETE"}
```

MQTT Topic Design (Final):

```
mixing/plant/{id}/
├ recipe      # HMI → PLC: Full recipe download (ALL steps)
├ cmd         # HMI → PLC: START, PAUSE, RESUME, ABORT
├ ack         # PLC → HMI: Recipe loaded / Command acknowledged
├ telemetry   # PLC → HMI: Real-time sensor data (every 1-2s)
├ status      # PLC → HMI: RUNNING, PAUSED, COMPLETE, ERROR
└ watchdog    # PLC → HMI: Heartbeat counter
```

Recipe JSON Schema:

```
{
  "batch_id": "P260411-02-01-001",
  "sku_id": "S77S743200",
  "sku_name": "Cafe Amazon",
  "batch_size": 1200.0,
  "plant_id": 1,
  "total_steps": 13,
  "total_phases": 7,
  "timestamp": "2026-04-15T16:40:00",
  "steps": [
    {
      "step_no": 10,
      "phase": "p0010",
      "phase_id": "A1010",
      "action_code": "10030",
      "re_code": "LS in Line",
      "destination": "1010",
      "require": 371.71,
      "uom": "kg",
      "low_tol": 0.001,
```

```
    "high_tol": 0.001,  
    "agitator_rpm": 80.0,  
    "high_shear_rpm": 0.0,  
    "temperature": 40.0,  
    "temp_low": 0.0,  
    "temp_high": 0.0,  
    "step_time": 0,  
    "step_timer_control": 0  
  }  
]  
}
```

6. Next Steps

- ☐ Implement recipe publish in `x61-MixingControl.vue` (Send All on Start)
- ☐ Update `plc_simulator.mjs` to receive & parse full recipe
- ☐ Add recipe ACK handling in `useMQTT.ts`
- ☐ Add step progress tracking on dashboard (current step highlight)

บทที่ 9: สถาปัตยกรรมระบบโดยรวม

Hybrid Architecture & Full Recipe Workflow

This document outlines the **Hybrid Communication Model** (CQRS) combined with the **Full Recipe Array** (DB1511) concept for the Mixing Control application.

1. System Architecture Diagram

```
graph TD
    subgraph Frontend [Vue.js Tablet UI]
        UI[Vue Components]
        MQTT_Sub[useMQTT.ts]
        API_Call[fetch / axios]
    end

    subgraph Backend [Python FastAPI]
        DB[(MySQL Database)]
        API[FastAPI Routes]
        Snap7[python-snap7]
    end

    subgraph Middleware [Telemetry Bridge]
        NR[Node-RED]
        MQ[RabbitMQ / MQTT]
    end

    subgraph PLC [Siemens S7-1200]
        DB1511[DB1511: Full Recipe Array]
        DB1512[DB1512: Live Telemetry]
        DB1513[DB1513: Handshake]
    end

    %% Telemetry Flow (Fast & Visual)
    DB1512 -->|Read| NR
    NR -->|Publish| MQ
    MQ -->|Subscribe| MQTT_Sub
    MQTT_Sub --> UI

    %% Command Flow (Strict & Transactional)
    UI ==>|1. HTTP POST /start-batch| API_Call
    API_Call ==> API
    API ==>|2. Validate & Read| DB
    API ==>|3. Write Full Array| Snap7
    Snap7 ==>|Direct S7 Write| DB1511
```

```
%% Handshake Feedback (Database Sync)
DB1513 -.->|Read End-of-Step| Snap7
Snap7 ==>|4. Update Logs| DB
```

2. Step-by-Step App Workflow

Phase A: Starting the Batch (The Command)

- Operator Action:** The operator scans a barcode and clicks “Start Production” on the Vue.js tablet.
- API Request:** The frontend sends an HTTP `POST /api/production/start` to FastAPI.
- Database Assembly:** FastAPI queries `v_sku_complete` to get all 20 steps for that specific SKU.
- Data Packing:** FastAPI uses `plc_interface.py` to pack all 20 steps into the binary array structure required by `type_FullRecipe`.
- Direct PLC Write:** FastAPI uses `snap7` to write the entire binary array directly to `DB1511` and sets `Cmd_LoadRecipe = True`.
- Confirmation:** FastAPI responds to the frontend with `200 OK`. (If the PLC is offline, FastAPI blocks the start and returns a 500 error).

Phase B: Mixing Execution (The Telemetry)

- PLC Takes Over:** The PLC sees `Cmd_LoadRecipe = True`, loads the array, and begins executing `Active_Step = 1`.
- Telemetry Streaming:** Node-RED continuously reads `DB1512` every 500ms and publishes to MQTT.
- UI Updates:** The Vue.js frontend receives the MQTT payload and updates the temperature gauges, active row highlighting, and timer *instantly*.

Phase C: Step Completion (The Handshake)

- PLC Finishes Step:** The PLC finishes Step 1, pulses `DB1513.Step_Complete`, and instantly moves to Step 2.
- Python Background Worker:** A background task in FastAPI (polling `DB1513` via `snap7`) detects the pulse.
- Database Logging:** FastAPI logs the final weight, temperature, and end-time of Step 1 into the SQL database.

3. Required Application Modules

To build this, you need to structure your app modules like this:

Python Backend (FastAPI)

- `plc_interface.py` (Update Needed):
 - Add a new Pydantic model for `DB1511_FullRecipe` and `DB1511_RecipeStep`.

- Write the `.serialize()` method to convert the array of steps into S7 byte format.

- **plc_service.py (New):**

- Use `python-snap7` to manage the TCP connection to the PLC.
- Function: `write_recipe_to_plc(batch_id, steps_array)`

- **worker_handshake.py (New):**

- An `asyncio` background loop that reads `DB1513` (Handshake) every 1 second and writes step completion data to the SQL database.

- **routes/production.py :**

- Endpoint for `POST /start-batch` that triggers the PLC write.

Node-RED (Middleware)

- **Remove DB1510 Logic:** Delete all nodes that currently write to DB1510. Node-RED is no longer responsible for sending recipes.
- **Keep DB1512 Logic:** Keep the S7-Read nodes for DB1512 and publish them to MQTT exactly as they are.

Vue.js Frontend (x3101-0110-frontEnd)

- **x61-MixingControl.vue :**

- When pressing start/next, instead of publishing an MQTT message, call an Axios HTTP POST request to your FastAPI backend.

- **useMQTT.ts :**

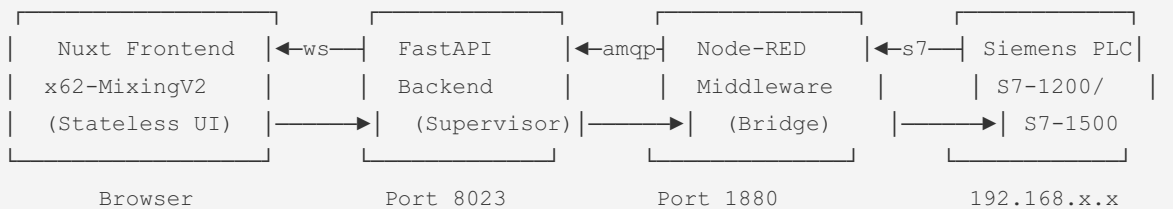
- Keep this exactly as it is! It will continue to listen to the telemetry stream to update the UI perfectly.

บทที่ 10: คู่มือติดตั้ง Deploy ระบบ

Production Deployment Guide — PLC Mixing Control V2

This document explains **exactly** what needs to happen on each layer of the system to move from our tested mock environment to a real, running production line.

Architecture Overview



Data Flow: 1. **Download:** Nuxt → FastAPI → Node-RED → PLC (write recipe once) 2. **Telemetry:** PLC → Node-RED (poll 500ms) → RabbitMQ → FastAPI → Nuxt (live state) 3. **Heartbeat:** FastAPI → RabbitMQ → Node-RED → PLC (toggle bit every 1s)

PART A: PLC Code (TIA Portal)

Important: All code below is in SCL (Structured Control Language) for Siemens TIA Portal V16+. Your PLC engineer should create these in the TIA Portal project.

A1. User-Defined Types (UDTs)

Create these two UDTs first. They define the shape of every recipe step and the overall recipe.

```
// =====
// UDT: "UDT_ProcessStep"
// Size: ~136 bytes per step
// =====
TYPE "UDT_ProcessStep"
VERSION : 0.1
STRUCT
    StepNo      : Int;           // Sub-step number (1, 2, 3...)
    ActionCode   : Int;           // Action code (10010=Setup, 21010=Manual Add, etc.)
```

```

ReCode      : String[25];      // Recipe ingredient code
SapCode     : String[20];     // SAP Material code
Destination : Int;           // Target vessel (0=MixTank, 1=Hopper...)
Require     : Real;           // Required weight (kg)
LowTol      : Real;           // Low tolerance (kg)
HighTol     : Real;           // High tolerance (kg)
Temperature : Real;           // Setpoint temperature (°C)
TempLow     : Real;           // Temp low limit
TempHigh    : Real;           // Temp high limit
AgitatorRPM : Real;           // Agitator speed setpoint (RPM)
HighShearRPM : Real;          // High-shear mixer speed (RPM)
StepTime    : Int;            // Step duration (seconds)
StepTimerCtl : Int;           // Timer control mode (0=none, 1=auto, 2=manual)
SetupStep   : Int;            // Setup step flag
Condition   : Int;            // Step condition code
QcTemp      : Bool;           // Requires QC temperature check
RecordSteam : Bool;           // Record steam pressure
RecordCTW   : Bool;           // Record CTW data
BrixRecord  : Bool;           // Requires Brix recording
PhRecord    : Bool;           // Requires pH recording
MasterStep  : Bool;           // Is a master step (phase start)
StepActive  : Bool;           // Step has valid data (not empty)
BrixSP      : Real;           // Brix setpoint
PhSP        : Real;           // pH setpoint

END_STRUCT;
END_TYPE

```

```

// =====
// UDT: "UDT_Process" (Phase)
// Each process contains up to 8 steps
// =====
TYPE "UDT_Process"
VERSION : 0.1
STRUCT
    ProcessNo      : Int;           // Phase number (10, 20, 30...)
    PhaseID        : Int;           // Phase ID from DB
    StepCount      : Int;           // Number of active steps in this phase
    ProcessActive  : Bool;           // Phase has valid data
    Steps          : Array[0..7] of "UDT_ProcessStep";
END_STRUCT;
END_TYPE

```

A2. Data Block — DB1780 “DB_RecipeData”

This is the main recipe data block that the PC writes to and the PLC reads from.

```

// =====
// DB1780: "DB_RecipeData"
// The PC writes recipe data here BEFORE starting

```



```
// =====
DATA_BLOCK "DB_RecipeData"
{ S7_Optimized_Access := 'FALSE' }
VERSION : 0.1
STRUCT
    // — Header (written by PC) —
    PlanID      : String[30];
    BatchID     : String[20];
    SkuID       : String[20];
    SkuName     : String[50];
    PlantID     : Int;
    BatchSize   : Real;
    ProcessCount : Int;          // Number of active phases

    // — Control Flags (written by PC) —
    RecipeReady : Bool;          // PC sets TRUE after download complete
    StartCmd    : Bool;          // PC sets TRUE to start batch
    PauseCmd    : Bool;          // PC sets TRUE to pause
    AbortCmd    : Bool;          // PC sets TRUE to emergency stop
    PC_Heartbeat : Bool;         // PC toggles every 1 second

    // — Status (written by PLC) —
    PLC_State    : Int;          // 0=Idle, 1=Running, 2=WaitManual, 3=Paused, 4=Done,
9=Error
    Current_Process : Int;       // Current active phase index (0-31)
    Current_Step    : Int;       // Current active step index within phase (0-7)
    Current_Step_Flat : Int;      // Flat step counter across all phases (1-based)
    Step_Timer_Act  : Int;       // Remaining seconds on current step timer
    Step_Complete   : Bool;      // Pulse: PLC sets TRUE when a step finishes
    Batch_Complete  : Bool;      // TRUE when all steps are done
    Error_Code      : Int;       // 0=None, 1=Heartbeat Lost, 2=Timeout

    // — Actuals (written by PLC from I/O) —
    MixTank_Temp_Act : Real;      // Actual tank temperature
    MixTank_Weight_Act: Real;     // Actual tank weight
    Agitator_Act     : Real;      // Actual agitator RPM
    HighShear_Act    : Real;      // Actual high-shear RPM
    Hopper_Weight_Act : Real;     // Actual hopper weight
    Brix_Act         : Real;      // Actual Brix reading
    PH_Act           : Real;      // Actual pH reading

    // — Recipe Array (32 phases × 8 steps) —
    Processes       : Array[0..31] of "UDT_Process";
END_STRUCT;
BEGIN
END_DATA_BLOCK
```

CRITICAL: Set `S7_Optimized_Access := 'FALSE'` so Node-RED can read/write using absolute byte offsets (e.g., `DB1780, DBD100`).

A3. Function Block — FB1780 “FB_MixingSequencer”

This is the **brain** of the system. It runs in a cyclic OB (e.g., OB1 or OB35) and manages the step-by-step execution.

```
// =====  
// FB1780: "FB_MixingSequencer"  
// Main sequencer logic – runs in OB1 every scan cycle  
// =====  
FUNCTION_BLOCK "FB_MixingSequencer"  
VERSION : 0.1  
  
VAR_INPUT  
    // Analog inputs from field instruments  
    Act_MixTank_Temp    : Real;  
    Act_MixTank_Weight  : Real;  
    Act_Agitator_RPM    : Real;  
    Act_HighShear_RPM   : Real;  
    Act_Hopper_Weight   : Real;  
    Act_Brix            : Real;  
    Act_PH              : Real;  
END_VAR  
  
VAR_OUTPUT  
    // Analog outputs to field devices  
    SP_Agitator_RPM     : Real;  
    SP_HighShear_RPM    : Real;  
    SP_Temperature      : Real;  
END_VAR  
  
VAR  
    stepTimer           : TON;           // IEC Timer for step duration  
    heartbeatTimer      : TON;           // Heartbeat watchdog timer  
    lastHeartbeat       : Bool;          // Previous heartbeat state  
    flatStepCounter     : Int;           // Running count across all phases  
END_VAR  
  
VAR_TEMP  
    curProcess          : Int;  
    curStep             : Int;  
    stepData            : "UDT_ProcessStep";  
    stepTimeSec         : Time;  
END_VAR
```

```

BEGIN

// === 1. UPDATE ACTUALS ===
"DB_RecipeData".MixTank_Temp_Act    := Act_MixTank_Temp;
"DB_RecipeData".MixTank_Weight_Act  := Act_MixTank_Weight;
"DB_RecipeData".Agitator_Act        := Act_Agitator_RPM;
"DB_RecipeData".HighShear_Act       := Act_HighShear_RPM;
"DB_RecipeData".Hopper_Weight_Act   := Act_Hopper_Weight;
"DB_RecipeData".Brix_Act             := Act_Brix;
"DB_RecipeData".PH_Act              := Act_PH;

// === 2. HEARTBEAT WATCHDOG ===
// If PC_Heartbeat stops toggling for 5 seconds → PC is dead
IF "DB_RecipeData".PC_Heartbeat <> #lastHeartbeat THEN
    #lastHeartbeat := "DB_RecipeData".PC_Heartbeat;
    #heartbeatTimer(IN := FALSE, PT := T#5s); // Reset timer
END_IF;
#heartbeatTimer(IN := TRUE, PT := T#5s);

IF #heartbeatTimer.Q AND "DB_RecipeData".PLC_State = 1 THEN
    // PC lost! But don't stop the batch – just log the error
    "DB_RecipeData".Error_Code := 1; // Heartbeat lost
    // If current step is manual (ActionCode 21010), pause
    #curProcess := "DB_RecipeData".Current_Process;
    #curStep    := "DB_RecipeData".Current_Step;
    #stepData   := "DB_RecipeData".Processes[#curProcess].Steps[#curStep];
    IF #stepData.ActionCode = 21010 THEN
        "DB_RecipeData".PLC_State := 2; // WaitManual – safe pause
    END_IF;
    // Automatic steps (mixing, heating) CONTINUE running
END_IF;

// === 3. COMMAND HANDLING ===
// Start
IF "DB_RecipeData".StartCmd AND "DB_RecipeData".RecipeReady THEN
    IF "DB_RecipeData".PLC_State = 0 OR "DB_RecipeData".PLC_State = 3 THEN
        "DB_RecipeData".PLC_State := 1; // Running
        "DB_RecipeData".StartCmd  := FALSE;
        "DB_RecipeData".Error_Code := 0;
        IF "DB_RecipeData".Current_Process = 0 AND "DB_RecipeData".Current_Step = 0 THEN
            #flatStepCounter := 1;
        END_IF;
    END_IF;
END_IF;

// Pause
IF "DB_RecipeData".PauseCmd THEN
    "DB_RecipeData".PLC_State := 3; // Paused
    "DB_RecipeData".PauseCmd  := FALSE;
END_IF;

```

```

// Abort
IF "DB_RecipeData".AbortCmd THEN
    "DB_RecipeData".PLC_State      := 0; // Idle
    "DB_RecipeData".AbortCmd      := FALSE;
    "DB_RecipeData".Current_Process := 0;
    "DB_RecipeData".Current_Step   := 0;
    #flatStepCounter := 0;
    SP_Agitator_RPM   := 0.0;
    SP_HighShear_RPM := 0.0;
    SP_Temperature    := 0.0;
END_IF;

// == 4. MAIN SEQUENCER (only when Running) ==
IF "DB_RecipeData".PLC_State = 1 THEN

    #curProcess := "DB_RecipeData".Current_Process;
    #curStep    := "DB_RecipeData".Current_Step;

    // Safety: check bounds
    IF #curProcess > 31 OR NOT "DB_RecipeData".Processes[#curProcess].ProcessActive THEN
        "DB_RecipeData".PLC_State      := 4; // Done
        "DB_RecipeData".Batch_Complete := TRUE;
        SP_Agitator_RPM   := 0.0;
        SP_HighShear_RPM := 0.0;
        RETURN;
    END_IF;

    // Get current step data
    #stepData := "DB_RecipeData".Processes[#curProcess].Steps[#curStep];

    // — Apply setpoints to outputs —
    SP_Agitator_RPM   := #stepData.AgitatorRPM;
    SP_HighShear_RPM := #stepData.HighShearRPM;
    SP_Temperature    := #stepData.Temperature;

    // — Update flat counter for UI —
    "DB_RecipeData".Current_Step_Flat := #flatStepCounter;

    // — MANUAL STEP (ActionCode 21010): Wait for operator —
    IF #stepData.ActionCode = 21010 THEN
        "DB_RecipeData".PLC_State := 2; // WaitManual
        // The PC will set StartCmd=TRUE again when operator scans/confirms
        RETURN;
    END_IF;

    // — TIMED STEP: Run step timer —
    IF #stepData.StepTime > 0 THEN
        #stepTimeSec := INT_TO_TIME(#stepData.StepTime * 1000);
        #stepTimer(IN := TRUE, PT := #stepTimeSec);
    END_IF;

```

```

        "DB_RecipeData".Step_Timer_Act :=
            TIME_TO_INT(#stepTimeSec - #stepTimer.ET) / 1000;

        IF NOT #stepTimer.Q THEN
            RETURN; // Still counting, come back next scan
        END_IF;
        // Timer finished - reset and advance
        #stepTimer(IN := FALSE, PT := #stepTimeSec);
    END_IF;

    // — STEP COMPLETE → advance —
    "DB_RecipeData".Step_Complete := TRUE; // Pulse for Node-RED to catch

    // Move to next step in this phase
    IF #curStep + 1 < "DB_RecipeData".Processes[#curProcess].StepCount THEN
        "DB_RecipeData".Current_Step := #curStep + 1;
        #flatStepCounter := #flatStepCounter + 1;
    ELSE
        // Move to next phase
        "DB_RecipeData".Current_Step := 0;
        "DB_RecipeData".Current_Process := #curProcess + 1;
        #flatStepCounter := #flatStepCounter + 1;

        // Check if we've exceeded all active phases
        IF #curProcess + 1 >= "DB_RecipeData".ProcessCount THEN
            "DB_RecipeData".PLC_State := 4; // Done
            "DB_RecipeData".Batch_Complete := TRUE;
            SP_Agitator_RPM := 0.0;
            SP_HighShear_RPM := 0.0;
        END_IF;
    END_IF;

END_IF;

END_FUNCTION_BLOCK

```

A4. Calling FB1780 in OB1

In your main program (OB1), create an instance DB and call the function block:

```

// In OB1 (Main Program)
"FB_MixingSequencer_DB"(
    Act_MixTank_Temp := "AI_MixTank_TT01", // Your analog input tag
    Act_MixTank_Weight := "AI_MixTank_WT01",
    Act_Agitator_RPM := "AI_Agitator_Speed",
    Act_HighShear_RPM := "AI_HighShear_Speed",
    Act_Hopper_Weight := "AI_Hopper_WT01",
    Act_Brix := "AI_Brix_Sensor",
    Act_PH := "AI_PH_Sensor",
    SP_Agitator_RPM => "AO_Agitator_SP", // Your analog output tag

```

```
SP_HighShear_RPM    => "AO_HighShear_SP",
SP_Temperature       => "AO_Temperature_SP"
);
```

Note: Replace "AI_xxx" and "AO_xxx" with your actual I/O tag names from the TIA Portal hardware configuration.

PART B: Node-RED Configuration

B1. Install Required Nodes

```
cd ~/.node-red
npm install node-red-contrib-s7
npm install node-red-contrib-amqp
```

B2. S7 Connection Configuration

In Node-RED, create an S7 connection: | Setting | Value | | Host | 192.168.1.1 (your PLC) | | Port | 102 | | Rack | 0 | | Slot | 1 | | Cycle Time | 500 ms |

B3. S7 Variable Mapping (Read from PLC → Publish to RabbitMQ)

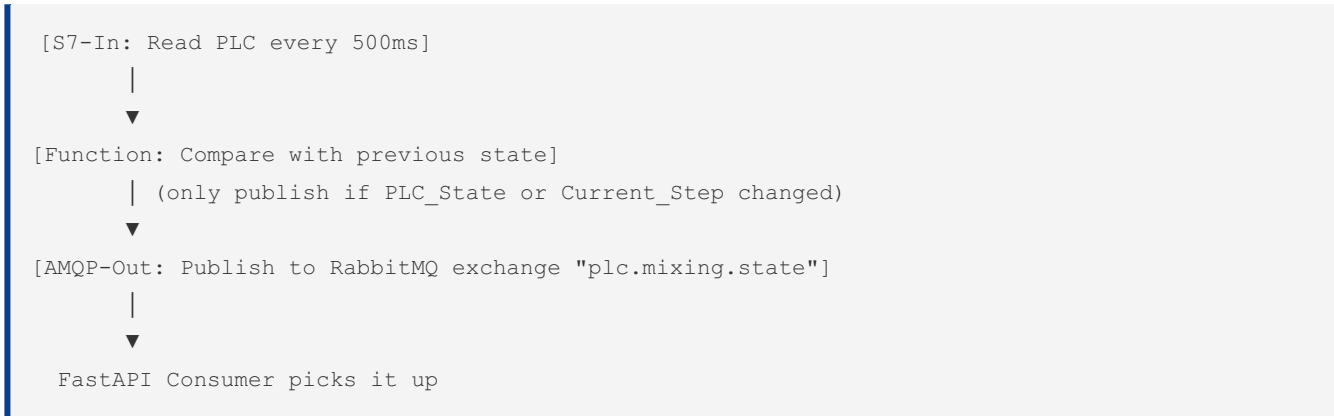
Create an **S7-In** node that reads these DB1780 addresses every 500ms:

Variable Name	S7 Address	Type
PLC_State	DB1780,INT18	INT
Current_Process	DB1780,INT20	INT
Current_Step	DB1780,INT22	INT
Current_Step_Flat	DB1780,INT24	INT
Step_Timer_Act	DB1780,INT26	INT
Step_Complete	DB1780,X28.0	BOOL
Batch_Complete	DB1780,X28.1	BOOL
Error_Code	DB1780,INT30	INT
MixTank_Temp_Act	DB1780,REAL32	REAL
MixTank_Weight_Act	DB1780,REAL36	REAL

Variable Name	S7 Address	Type
Agitator_Act	DB1780,REAL40	REAL
HighShear_Act	DB1780,REAL44	REAL
Hopper_Weight_Act	DB1780,REAL48	REAL
Brix_Act	DB1780,REAL52	REAL
PH_Act	DB1780,REAL56	REAL
BatchID	DB1780,S34.20	STRING

Important: The exact byte offsets above are **approximate**. After you create DB1780 in TIA Portal with `Optimized Access = FALSE`, open the DB and look at the **Offset** column. Use those exact values in Node-RED.

B4. Node-RED Flow Logic



Function Node (change detection):

```
// Store previous state in flow context
var prev = flow.get('plc_state') || {};
var cur = msg.payload;

// Only publish if something changed
if (prev.PLC_State !== cur.PLC_State ||
    prev.Current_Step_Flat !== cur.Current_Step_Flat ||
    prev.Step_Complete !== cur.Step_Complete) {

    flow.set('plc_state', cur);
    msg.payload = {
        PLC_State: cur.PLC_State,
        Current_Process: cur.Current_Process,
```

```

        Current_Step: cur.Current_Step,
        Current_Step_Flat: cur.Current_Step_Flat,
        Step_Timer_Act: cur.Step_Timer_Act,
        Step_Complete: cur.Step_Complete,
        Batch_Complete: cur.Batch_Complete,
        BatchID: cur.BatchID,
        MixTank_Temp_Act: cur.MixTank_Temp_Act,
        MixTank_Weight_Act: cur.MixTank_Weight_Act,
        Agitator_Act: cur.Agitator_Act,
        HighShear_Act: cur.HighShear_Act,
        timestamp: new Date().toISOString()
    };
    return msg;
}
return null; // Don't publish if nothing changed

```

B5. Writing Recipe TO PLC (PC → Node-RED → PLC)

Create an **HTTP-In** node (POST `/api/plc/write-recipe`) that accepts the JSON recipe from FastAPI and writes it to DB1780 using S7-Out nodes.

Flow:

```

[HTTP-In: POST /api/plc/write-recipe]
  |
  ▼
[Function: Map JSON to S7 writes]
  |
  ▼
[S7-Out: Write header fields]
[S7-Out: Write Process[0].Steps[0..7]]
[S7-Out: Write Process[1].Steps[0..7]]
... (loop through all active processes)
  |
  ▼
[S7-Out: Write RecipeReady = TRUE]
  |
  ▼
[HTTP-Response: 200 OK]

```

B6. Heartbeat Relay (PC → PLC)

```

[AMQP-In: Subscribe "pc.heartbeat"]
  |
  ▼
[S7-Out: Write DB1780.PC_Heartbeat (toggle TRUE/FALSE)]

```


PART C: Production Deployment Checklist

Step 1: PLC Setup (TIA Portal)

- ☐ Create `UDT_ProcessStep` and `UDT_Process` in TIA Portal
- ☐ Create `DB1780` with `Optimized Access = FALSE`
- ☐ Create `FB1780` with the sequencer logic
- ☐ Call `FB1780` in OB1 with correct I/O mappings
- ☐ Download to PLC and verify DB1780 appears in the watch table
- ☐ Note the **exact byte offsets** from TIA Portal's DB view

Step 2: Node-RED Setup

- ☐ Install `node-red-contrib-s7` and `node-red-contrib-amqp`
- ☐ Configure S7 connection to PLC IP (test with a simple read first)
- ☐ Map the S7 variables using exact offsets from Step 1
- ☐ Create the polling flow (read every 500ms)
- ☐ Create the change-detection function node
- ☐ Create the RabbitMQ publish node
- ☐ Create the recipe-write HTTP endpoint
- ☐ Create the heartbeat relay flow
- ☐ **Test:** Read `PLC_State` from Node-RED debug panel

Step 3: FastAPI Backend

- ☐ Update `POST /plc/send-recipe/{batch_id}` to POST the JSON to Node-RED's HTTP endpoint instead of returning it
- ☐ Add RabbitMQ consumer to listen for `plc.mixing.state` events
- ☐ On each event, write to `mixing_batch_step_log` table
- ☐

Add WebSocket broadcast to push state to frontend

- ☐ Add heartbeat publisher (toggle `pc.heartbeat` every 1s)

Step 4: Frontend (Already Done)

- ☒ `x62-MixingControlV2.vue` is stateless
- ☒ Reacts to `plantData` telemetry updates
- ☒ Recovery on mount works
- ☒ `downloadRecipeToPlc()` calls the backend API

Step 5: Integration Test

- ☐ Load a test batch on the PLC via the UI
- ☐ Verify recipe appears in DB1780 watch table
- ☐ Click START → verify PLC_State changes to 1
- ☐ Watch steps auto-advance in the UI
- ☐ **Pull-the-plug test:** Kill FastAPI at Step 5, restart, verify UI recovers at the correct step
- ☐ **Manual step test:** Verify PLC pauses at ActionCode 21010 and waits for operator confirmation

PART D: Action Code Reference for PLC Logic

The PLC sequencer needs to know which steps are automatic and which require manual intervention:

Action Code	Description	PLC Behavior
10010	Setup / Initialize	Auto — apply setpoints, no wait
10020	Heating	Auto — wait for temp + time
10030	Mixing / Blending	Auto — run agitator for time
10040	Pasteurize	Auto — hold temp for time
21010	Manual Add to Mix Tank	MANUAL — PLC waits for PC
30020	Manual Transfer	MANUAL — PLC waits for PC

Action Code	Description	PLC Behavior
40010	QC Check / Record	MANUAL — requires Brix/pH

For all 21010 and 30020 steps, the PLC sets `PLC_State = 2` (WaitManual) and holds position until the PC sends `StartCmd = TRUE` after the operator confirms.

PART E: Network Requirements

Device	IP Address	Port	Protocol
PLC	192.168.1.1	102	S7 (TCP)
Node-RED	192.168.1.100	1880	HTTP
RabbitMQ	192.168.1.100	5672	AMQP
FastAPI	192.168.1.100	8023	HTTP
Nuxt	192.168.1.100	3000	HTTP/WS

All middleware (Node-RED, RabbitMQ, FastAPI, Nuxt) can run on a **single industrial PC**. Only the PLC needs a separate Ethernet connection.

PART F: Safety Considerations

- Emergency Stop:** The PLC's hardware E-Stop circuit must be independent of this software. The `AbortCmd` is a software-level stop only.
- Heartbeat Timeout:** If `PC_Heartbeat` stops for 5 seconds, the PLC continues automatic steps but pauses manual steps. This prevents product loss.
- Recipe Validation:** Before setting `RecipeReady = TRUE`, the PLC should verify `ProcessCount > 0` and `BatchID` is not empty.
- Watchdog:** Node-RED should monitor its own S7 connection. If the S7 link drops, it should publish an alarm to RabbitMQ so FastAPI can alert the operator.

บทที่ 11: ตารางสัญญาณเตือน (Alarm List) และการแก้ไขปัญหา (Troubleshooting Guide)

11.1 สัญญาณเตือนจากระบบทั้งหมด (Complete Alarm List)

ระดับความรุนแรง (Severity Levels)

ระดับ	สัญลักษณ์	ความหมาย	การกระทำ
Critical		ระบบหยุดทำงาน / ข้อมูลสูญหาย	หยุดผลิตทันที แจ้งวิศวกร
Warning		มีความผิดปกติ แต่ยังทำงานต่อได้	ตรวจสอบและแก้ไขเร็วๆ
Info		แจ้งสถานะปกติ	รับทราบเท่านั้น
Success		ดำเนินการสำเร็จ	ไม่ต้องทำอะไร

11.1.1 Alarm หน้าจอ Mixing Control (x61)

รหัส / ข้อความ	ระดับ	สาเหตุ	วิธีแก้ไข
สแกนผิด Phase!		สแกนวัตถุดิบของ Phase ที่ยังไม่ถึงหรือผ่านไปแล้ว	ตรวจสอบป้าย Phase บนถาด และสแกนให้ถูก Phase ที่ระบบแสดงอยู่
WRONG BOX!	Critical	สแกนวัตถุดิบที่ไม่ใช่ของ Batch นี้เลย (ผิด Lot/ กล่อง)	หยุดทันที! ตรวจสอบว่าหยิบกล่องวัตถุดิบถูกต้องหรือไม่ แจ้งหัวหน้า
INTERLOCK — Step X blocked!		ค่า Setpoint ไม่ผ่านเงื่อนไข หรือสแกนยังไม่ครบ	ตรวจสอบว่าสแกนวัตถุดิบครบทุกตัวใน Phase แล้วรอ PLC ตอบกลับ
PLC is offline! Cannot send command.		PLC ขาดการเชื่อมต่อ	ตรวจสอบสาย LAN, รีเซ็ต rtk xmixing-backend.service
Failed to download recipe to PLC		ส่งสูตรไป PLC ไม่สำเร็จ	กด Refresh แล้วลองใหม่ หรือตรวจ Network
QC Data Required! Please fill in Brix/pH		ถึงขั้นตอนวัดคุณภาพ ต้องกรอกค่าก่อนดำเนินการต่อ	วัดค่า Brix/pH แล้วกรอกลงช่อง กด Confirm QC

รหัส / ข้อความ	ระดับ	สาเหตุ	วิธีแก้ไข
QC save failed		บันทึกค่า QC ไม่สำเร็จ (Network/Server)	ตรวจสอบการเชื่อมต่อ Internet/ Server แล้วกด Save ใหม่
Scanned volume out of tolerance!		ปริมาณที่อ่านได้จาก QR Code เกิน Tolerance ที่กำหนด	ตรวจสอบน้ำหนักจริงบนเครื่องชั่ง และสแกน QR Code ใหม่
Weight (Require) not in range!		น้ำหนักวัตถุดิบที่ชั่งได้ไม่อยู่ในเกณฑ์	ชั่งวัตถุดิบใหม่ให้ได้น้ำหนักตาม Require WT
Process parameters out of tolerance		ค่า Process (อุณหภูมิ/ RPM) เกินขอบเขต Software Interlock	รอค่า Actual มาอยู่ในเกณฑ์ หรือตรวจสอบเซ็นเซอร์
No active batch in PLC DB15X1		ไม่มีสูตรใน PLC หรือ Batch ยังไม่ถูกส่งไป	กดปุ่ม Download Recipe to PLC ใหม่
Failed to read PLC DB15X1 or no data		อ่านข้อมูลจาก PLC ไม่ได้	รอ 3-5 วิ แล้ว Refresh หรือแจ้งวิศวกร
No SKU steps found		ไม่มีข้อมูลสูตรการผลิตในระบบ	ตรวจสอบว่าเลือก Batch และ SKU ถูกต้อง
Manual Override applied		Supervisor บังคับผ่าน Step โดยใส่เหตุผล	บันทึกเหตุผลในระบบเรียบร้อยแล้ว
Production STARTED!		เริ่มการผลิตสำเร็จ	PLC รับสูตรและเริ่มทำงาน
BATCH COMPLETE!		ผลิตครบทุกขั้นตอนแล้ว	ดำเนินการล้างถัง/เตรียม Batch ถัดไป
QC Data Saved!		บันทึกค่า QC สำเร็จ	ระบบเดินต่ออัตโนมัติ

11.1.2 Alarm หน้าจอ Check for Production (x60)

ข้อความ	ระดับ	สาเหตุ	วิธีแก้ไข
FIFO ORDER VIOLATION		ใช้วัตถุดิบไม่ตามลำดับ FIFO (ใช้ล็อตใหม่ก่อนของเก่า)	ตรวจสอบลำดับล็อตวัตถุดิบ และใช้ตาม FIFO ที่ระบบแนะนำ
User not found		สแกนบัตรพนักงานไม่พบในระบบ	ตรวจสอบว่าพนักงานถูก Register ในระบบ (Admin ตรวจที่ x89)

ข้อความ	ระดับ	สาเหตุ	วิธีแก้ไข
Cannot reach server to verify user	.	Server ขาดการเชื่อมต่อ	แจ้งทีม IT ตรวจสอบ xmixing-backend.service
Hold action failed	.	สั่ง Hold Batch ไม่สำเร็จ	ลอง Refresh และสั่ง Hold ใหม่
Failed to verify recipe transfer to PLC	.	ส่งสูตรไป PLC ล้มเหลว	ตรวจสอบ Network ระหว่าง Server และ PLC
Failed to update status	.	อัปเดตสถานะ Batch ไม่สำเร็จ	ตรวจสอบ Server แล้ว Refresh
Failed to force verify ingredient	.	Verify วัตถุดิบแบบ Force ไม่สำเร็จ	ต้องมีสิทธิ์ Supervisor หรือ ตรวจสอบ Network
No bags have been packed for this batch yet	.	ยังไม่มีกราสแกนถุงวัตถุดิบเลย	ให้ห้อง Prebatch ชั่งและสแกนถุงก่อน
PLEASE SCAN PRE-BATCH LABELS	.	ระบบรอการสแกน Label ถุงวัตถุดิบ	ใช้ปืนสแกนยิง Label ถุงที่ชั่งแล้ว
Batch Loaded – SCAN PREBATCH LABELS	.	โหลด Batch สำเร็จ รอสแกน	ดำเนินการสแกน Label
ALL INGREDIENTS VERIFIED!		ยืนยันวัตถุดิบครบทุกรายการแล้ว	Batch พร้อมส่งผลิต กด GO TO MIXING
X ingredient(s) approved		Verify วัตถุดิบสำเร็จ	ดำเนินการรายการถัดไป

11.2 การแก้ไขปัญหาเบื้องต้น (Troubleshooting Guide)

11.2.1 ปัญหาด้านการเชื่อมต่อ (Connectivity Issues)

ปัญหาที่พบ	วิธีตรวจสอบ	วิธีแก้ไข
หน้าจอขึ้น PLC OFFLINE	ตรวจสอบสัญญาณไฟ LAN Switch, ลอง Ping IP PLC	ตรวจสอบสาย LAN หลังจอ HMI, รีสตาร์ท xmixing-backend service
ข้อมูล Gauge ไม่อัปเดต (หยุดนิ่ง)	ดูที่ไฟ Watchdog indicator บนหน้าจอ	กด Refresh (F5) หรือรีสตาร์ท Node-RED service

ปัญหาที่พบ	วิธีตรวจสอบ	วิธีแก้ไข
Login สำเร็จแต่ข้อมูลไม่โหลด	ตรวจสอบ Network ระหว่าง HMI และ Server	แจ้ง IT รีเซ็ตอาร์ก xmixing-backend
ส่งคำสั่ง START แล้ว PLC ไม่ตอบสนอง	ตรวจสอบ DB1511 ใน PLC Watch Table	ลอง Download Recipe ใหม่ แล้วกด START อีกครั้ง

11.2.2 ปัญหาการสแกน (Scanning Issues)

ปัญหาที่พบ	วิธีตรวจสอบ	วิธีแก้ไข
สแกนแล้วไม่มีอะไรเกิดขึ้น	ตรวจสอบว่าเป็นสแกนเชื่อมต่ออยู่	ลองถอด-เสียบ USB ปีนสแกน แล้วสแกนใหม่
ขึ้น “Scan - no volume”	QR Code อ่านได้บางส่วน volume หาย	สแกนใหม่จ่อใกล้ๆ ให้อ่านครบ
ขึ้น “สแกนผิด Phase”	ดูว่า Phase ปัจจุบันในหน้าจอคืออะไร	วางถุงนั้นไว้ก่อน หยิบถุงที่ Phase ตรงกัน
ขึ้น “WRONG BOX”	ตรวจสอบ Batch ID บนถุงกับหน้าจอ	หยุดการเติม แจ้งหัวหน้า ตรวจสอบวัตถุดิบ
FIFO Violation	ตรวจสอบวันหมดอายุล็อตวัตถุดิบ	ใช้ล็อตเก่าก่อนเสมอ

11.2.3 ปัญหาการผลิต (Production Issues)

ปัญหาที่พบ	วิธีตรวจสอบ	วิธีแก้ไข
สแกนครบแล้ว ระบบไม่ Next Step	ตรวจสอบว่าสแกนครบทุกตัวใน Phase หรือยัง	รอ 3-5 วิ → Refresh → ถ้าค้างแจ้ง Supervisor กด Bypass
อุณหภูมิไม่ขึ้นตาม Setpoint	ดูค่า Actual Temp บนหน้าจอ	ตรวจสอบระบบ Steam/Heating Element แจ้งช่างไฟฟ้า
น้ำหนักในถังไม่ตรง	ดูค่า Actual Weight บนหน้าจอ	ตรวจสอบเซ็นเซอร์น้ำหนัก (Load Cell) แจ้งช่าง
กรอก QC แล้วบันทึกไม่ได้	ตรวจสอบ Network ระหว่าง HMI และ Server	แจ้ง IT ตรวจสอบ Database connection
Batch หยุดกลางคัน	ดูข้อความ Alarm ล่าสุดบนหน้าจอ	กด PAUSE รอวิศวกรตรวจสอบ ห้าม ABORT เองโดยไม่ได้รับอนุญาต

11.2.4 ขั้นตอนการกู้คืนหลังเหตุฉุกเฉิน (Emergency Recovery)

ลำดับ	การกระทำ	ผู้รับผิดชอบ
1	กด PAUSE เพื่อหยุดเครื่องจักรชั่วคราว	Operator
2	แจ้งหัวหน้ากะ และบันทึก Step/Phase ที่หยุดลงในใบบันทึก	Operator

ลำดับ	การกระทำ	ผู้รับผิดชอบ
3	ตรวจสอบสาเหตุ (Network / เซ็นเซอร์ / วัตถุดิบ)	Supervisor
4	แก้ไขปัญหา → กด RESUME เพื่อทำต่อจากจุดเดิม	Supervisor
5	หากทำต่อไม่ได้ กด BYPASS เพื่อข้าม Step	Supervisor
6	หากต้องทิ้ง Batch: กด ABORT → บันทึกเหตุผลในระบบ	Supervisor
7	ล้างถัง (CIP) ก่อนเริ่ม Batch ใหม่	Operator

11.3 การติดต่อทีมสนับสนุน (Support Contacts)

หน่วยงาน	ปัญหาที่ติดต่อได้	ช่องทาง
xDev.CO,LTD (ผู้พัฒนาระบบ)	Bug ในซอฟต์แวร์, พังค์ชันผิดปกติ	ช่องทางที่ตกลงกับ Mitr Phol
ฝ่าย IT Mitr Phol	Network, Server, Database	ติดต่อภายใน
ฝ่ายวิศวกรรม (Automation)	PLC, Sensor, Hardware, Node-RED	ช่างประจำโรงงาน
ฝ่ายผลิต (Production)	ขั้นตอนการทำงาน, สูตรการผลิต	หัวหน้ากะ